**SAVITRIBAI PHULE PUNE UNIVERSITY**

**A PROJECT REPORT ON**

# SPECTROGUARD: AI POWERED CYBER THREAT DETECTION

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

## BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

**SUBMITTED BY**


| STUDENT NAME | EXAM NO. |
|---|---|
| 1 .Pratiksha Dnyashwar Alhat | BEA02 |
| 2. Shraddha Sunil Gadekar | BEA21 |
| 3. Kapil Ganesh Waghumbare | BEB57 |





"Towards Ubiquitous Computing Technology"

## DEPARTMENT OF COMPUTER ENGINEERING

“Techno – Social Excellence”


**MARATHWADA MITRAMANDAL’S INSTITUTE OF TECHNOLOGY (MMIT)**
**LOHGAON, PUNE-411047 (2025-26)**




# CERTIFICATE

This is to certify that the project report entitled

**SPECTROGUARD: AI POWERED CYBER THREAT DETECTION**

Submitted by

| **STUDENT NAME** | **Exam No :** |
|---|---|
| 1 Pratiksha Dnyashwar Alhat | BEA02 |
| 2. Shraddha Sunil Gadekar | BEA21 |
| 3. Kapil Ganesh Waghumbare | BEB57 |

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. S. S. Chaudhari** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

**Dr. S. S. Chaudhari**
Guide
Department of Computer Engineering

**Dr. Subhash G. Rathod**
HOD
Department of Computer Engineering

**Dr. R. V. Bhortake**
Principal,
Marathwada Mitra Mandal's Institute of Technology Pune – 47

Signature of Internal Examiner
Place : Pune
Date :

Signature of External Examiner




**Institute**

# Vision

Techno-Social Excellence

# Mission

**M1:** Enhance technology transfer

**M2**: Implement entrepreneurship

**M3**: Promote global competency

**M4**: Integrate innovative pedagogy

**M5**: Create excellent human

# Program Educational Objectives (PEOs)

**PEO 1**: To apply the principle of mathematics, in area of computer science &engineering.

**PEO 2**: To design a system to meet global requirements of industry.

**PEO 3**: To produce graduates for solving ethical & social competency.

**PEO 4:** To produce a personal & professional leadership in Them workplace & community.

"Techno-social Excellence"

**Marathwada Mitra Mandal's**

**INSTITUTE OF TECHNOLOGY**

**Lohgaon, Pune-411047**

"येथे बहुतांचे हित"

"Towards ubiquitous Computing Technology"

## Department of Computer Engineering

# VISION

**"Towards Ubiquitous Computing Technology"**

# MISSION

- **Creating Industry Oriented Competency**
- **Innovating Computing Studies**
- **Inspiring Students to Endorse New Technologies**
- **Virtualizing Upcoming World for Implementation**

## Computer Engineering Department

### Course Outcomes:

**On completion of the course, student will–**

1. Identify, analyze, and solve problems creatively through sustained critical investigation by applying knowledge
2. Demonstrate skill and knowledge of current information, technological tools and techniques for societal and professional ethical standards.
3. Participate effectively oral, written and visual communication for integrate information from multiple sources and practice diligence, commitment to excellence needed to engage in

# ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on '**SPECTROGUARD: AI POWERED CYBER THREAT DETECTION**'.*

*We invested a lot of time and energy into this project. However, a lot of people contributed to and encouraged us as we finished our tasks. We want to express our sincere gratitude to each one of them.*

*We are appreciative of Dr. S.S. Chaudhari for serving as our mentor, being present at all times to oversee us, and giving the project data and insights that are required. We are appreciative of his guidance.*

*We would like to express our gratitude to our parents and friends at Marathwada Mitra Mandal's Institute of Technology for their support and co-operation in helping us finish this job.*

**NAME OF STUDENTS:**

1 Pratiksha Dnyashwar Alhat

2. Shraddha Sunil Gadekar

3. Kapil Ganesh Waghumbare

# ABSTRACT

In the modern digital era, the shift toward remote work and cloud-based communication has significantly increased the exposure of users to advanced and evolving cyber threats. Many individuals, however, remain unaware of these risks or lack the technical expertise to protect themselves effectively. **SEPCETO-GUARD** is an innovative, AI-driven cybersecurity toolkit developed to simplify digital safety and promote cyber awareness among everyday users.

The toolkit integrates multiple essential security modules, including a **password strength evaluator**, **port scanner**, **phishing site detector**, **open redirect checker**, **hash identifier**, **browser fingerprint viewer**, and a **local network risk assessment checklist**. Each tool is designed to analyze potential vulnerabilities, provide real-time threat insights, and guide users in mitigating risks efficiently.

Unlike traditional cybersecurity tools that demand technical proficiency, SEPCETO-GUARD focuses on **user-friendly interfaces** and **educational feedback**, helping individuals not only detect threats but also understand their nature and impact. By merging **AI-based detection mechanisms** with **interactive learning components**, the system encourages proactive security practices rather than reactive responses.

Ultimately, SEPCETO-GUARD bridges the gap between complex cybersecurity knowledge and practical digital defense, empowering users with minimal technical background to secure their devices, data, and networks confidently in an increasingly AI-driven threat landscape.

# TABLE OF CONTENTS


**i** LIST OF ABBREVIATIONS

**ii** LIST OF FIGURES

**iii** LIST OF TABLES

| SR.NO. | | CHAPTER TITLE | | PAGE NO. |
|---|---|---|---|---|
| **01** | | **Introduction** | | 1 |
| | 1.1 | Overview | | 2 |
| | 1.2 | Motivation | | 3 |
| | 1.3 | Problem Definition and Objectives | | 4 |
| | 1.4 | Project Scope & Limitations | | 6 |
| | 1.5 | Methodologies of Problem solving | | 8 |
| **02** | | **Literature Survey** | | 11 |
| | 2.1 | Online Survey | | 20 |
| **03** | | **Software Requirements Specification** | | 21 |
| | 3.1 | **Introduction** | | 22 |
| | | 3.1.1 | User Classes and Characteristics | 22 |
| | | 3.1.2 | Assumptions and Dependencies | 22 |
| | 3.2 | **Functional Requirements** | | 23 |
| | | 3.2.1 | System Feature | 23 |
| | 3.3 | **External Interface Requirements** | | 26 |
| | | 3.3.1 | User Interfaces | 26 |
| | | 3.3.2 | Application Interfaces | 26 |
| | | 3.3.3 | Security and Compliance Interfaces | 26 |
| | | 3.3.4 | Communication Interfaces | 26 |
| | 3.4 | **Nonfunctional Requirements** | | 27 |

| | | | | |
|---|---|---|---|---|
| | | 3.4.1 | Performance Requirements | 27 |
| | | 3.4.2 | Safety Requirements | 27 |
| | | 3.4.3 | Security Requirements | 27 |
| | | 3.4.4 | Software Quality Attributes | 28 |
| | 3.5 | **System Requirements** | | 29 |
| | | 3.5.1 | Database Requirements | 29 |
| | | 3.5.2 | Software Requirements (Platform Choice) | 29 |
| | | 3.5.3 | Hardware Requirements | 30 |
| | 3.6 | **Analysis Models: SDLC Model to be applied** | | 31 |
| | 3.7 | **System Implementation Plan** | | 33 |
| **04** | | **System Design** | | 35 |
| | 4.1 | System Architecture | | 36 |
| | 4.2 | Data Flow Diagrams | | 40 |
| | 4.3 | Entity Relationship Diagrams | | 41 |
| | 4.4 | UML Diagrams | | 43 |
| | 4.5 | Sequence Diagrams | | 45 |
| | 4.6 | Activity Diagrams | | 47 |
| | 4.7 | Class Diagram | | 49 |
| | 4.8 | Component Diagram | | 50 |
| | 4.9 | Deployment Diagrams | | 51 |
| | 4.10 | State Machine Diagram | | 52 |
| **05** | | **Project Plan** | | 53 |
| | 5.1 | **Project Estimate** | | 54 |
| | | 5.1.1 | Reconciled Estimates | 54 |
| | | 5.1.2 | Project Resources | 54 |
| | 5.2 | **Risk Management** | | 56 |

| | | | | |
|---|---|---|---|---|
| | | 5.2.1 | Risk Identification | 56 |
| | | 5.2.2 | Risk Analysis | 57 |
| | | 5.2.3 | Overview of Risk Mitigation, Monitoring, Management | 57 |
| | 5.3 | **Project Schedule** | | 59 |
| | | 5.3.1 | Project Task Set | 59 |
| | | 5.3.2 | Task Network | 60 |
| | | 5.3.3 | Timeline Chart | 60 |
| | 5.4 | **Team Organization** | | 62 |
| | | 5.4.1 | Team structure | 62 |
| | | 5.4.2 | Management reporting and communication | 62 |
| | | 5.4.3 | Communication Approach | 62 |
| **06** | | **Project Implementation** | | 63 |
| | 6.1 | Overview of Project Modules | | 64 |
| | 6.2 | Tools and Technologies Used | | 65 |
| | 6.3 | **Algorithm Details** | | 65 |
| | | 6.3.1 | Layered Markov Model Algorithm | 65 |
| | | 6.3.2 | Role-Based Access Control (RBAC) | 66 |
| **07** | | **Software Testing** | | 68 |
| | 7.1 | Types of Testing | | 69 |
| | 7.2 | Test cases & Test Results | | 69 |
| | 7.3 | Analytics of Project | | 70 |
| **08** | | **Results** | | 72 |
| | 8.1 | Outcomes | | 73 |
| | 8.2 | Screen Shots | | 74 |
| **09** | | **Conclusions** | | 77 |

| | | | |
|---|---|---|---|
| | 9.1 | Conclusion | 78 |
| | 9.2 | Applications | 79 |
| | 9.3 | Future Scope | 80 |
| **10** | | **References** | 81 |

## LIST OF ABBREVIATIONS

| ABBREVIATION | ILLUSTRATION |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| DB | Database |
| DNS | Domain Name System |
| DDoS | Distributed Denial of Service |
| GDPR | General Data Protection Regulation |
| HTML | HyperText Markup Language |
| CSS | Cascading Style Sheets |
| JS | JavaScript |
| MFA | Multi-Factor Authentication |
| OAuth | Open Authorization |
| RBAC | Role-Based Access Control |
| REST | Representational State Transfer |
| SRS | Software Requirements Specification |
| SSL | Secure Sockets Layer |
| TLS | Transport Layer Security |
| UI | User Interface |
| URL | Uniform Resource Locator |
| VPN | Virtual Private Network |
| IP | Internet Protocol |
| SHA-256 | Secure Hash Algorithm 256-bit |
| Phishing | Fraudulent attempt to obtain sensitive information |
| UAT | User Acceptance Testing |
| NOSQL | Not Only SQL |

## LIST OF FIGURES


| Figure No. | Illustration | Page No. |
|---|---|---|
| 3.1 | Waterfall Model | 31 |
| 4.1 | System Architecture | 36 |
| 4.1.1 | Password Strength checker | 37 |
| 4.1.2 | Phishing Site Checker | 38 |
| 4.1.3 | Hash ID Generator | 39 |
| 4.2 | DFD Level – 0 | 40 |
| 4.3 | ER Diagram | 41 |
| 4.4 | UML Diagram | 43 |
| 4.5 | Sequence Diagram | 45 |
| 4.6 | Activity Diagram | 47 |
| 4.7 | Class Diagram | 49 |
| 4.8 | Component Diagram | 50 |
| 4.9 | Deployment Diagram | 51 |
| 4.10 | State Machine Diagram | 52 |
| 5.1 | Timeline Chart Diagram | 60 |
| 8.2 | Screenshots | 74 |
| 8.2.5 | Code Output Snapshot | 76 |


## LIST OF TABLES


| Table | Illustration | Page No. |
|---|---|---|
| 2 | Literature Survey | 17 |
| 7.2 | Test Cases and Test Results | 69 |

# Chapter 1
# Introduction

# CHAPTER 1
# INTRODUCTION

## 1.1 OVERVIEW

In today's digital-first world, personal and professional activities are heavily dependent on online platforms, cloud services, and remote connectivity. While this shift has increased convenience and efficiency, it has also expanded the attack surface for cybercriminals. Threats such as phishing, credential theft, open-redirect attacks, and network exploitation continue to rise at alarming rates.

Most users remain unaware of how vulnerable their systems and data truly are. Weak passwords, unsecured Wi-Fi networks, and unverified websites are common gateways for hackers. Large organizations may have the resources to deploy advanced security systems, but individuals and small businesses often lack the expertise, budget, or awareness to protect themselves effectively.

**SpectroGuard** addresses this gap by offering a **web-based, AI-driven cybersecurity toolkit** that combines multiple essential tools in one platform. The system not only detects potential threats but also educates users on why these vulnerabilities exist and how to prevent them in the future. This dual focus on **detection and awareness** is the unique strength of SpectroGuard.

The toolkit includes the following features:

- **Password Strength Checker**: Evaluates password complexity and resistance to cracking attacks.
- **Phishing Site Checker:** Uses AI to detect fake or malicious websites.
- **Phishing URL Checker:** Scans and flags suspicious or unsafe links.
- **Hash ID Generator:** Identifies hash types to verify data integrity and security.

By combining these features in a lightweight, user-friendly environment, SpectroGuard enables proactive cybersecurity practices for both technical and non-technical users.

## 1.2 MOTIVATION

The motivation behind SpectroGuard arises from the growing frequency and sophistication of cyber threats in recent years. Statistics reveal that phishing attacks and credential theft account for a majority of successful breaches, and yet, most attacks could have been prevented with basic awareness and early detection.

For example:

- Studies show that over 80% of breaches involve weak or reused passwords.
- Public Wi-Fi networks expose users to man-in-the-middle (MITM) attacks.
- AI-generated phishing websites are nearly indistinguishable from legitimate ones, making detection harder for average users.
- Open ports often leave systems exposed to malware or unauthorized access.

Most users lack the tools and technical knowledge to protect themselves from such threats. Traditional enterprise-grade security systems (like SIEM tools, IDS/IPS, and firewalls) are too complex or costly for individuals or small organizations.

SpectroGuard is motivated by the idea of making cybersecurity accessible, proactive, and educational. By providing simple, AI-driven tools in one platform, it empowers ordinary users to protect themselves while also raising awareness of best practices in cyber hygiene.

## 1.3 PROBLEM DEFINITION AND OBJECTIVES

In the modern digital era, cyber threats are increasing at an unprecedented pace due to widespread internet connectivity, public Wi-Fi usage, remote access, and the rapid growth of online services. Unfortunately, most users remain unaware of their security vulnerabilities and lack access to tools that can help them identify risks before they are exploited.

Some common issues faced today include:

1. **Weak or Reused Passwords** – Studies reveal that over 80% of security breaches involve weak or compromised passwords.
2. **Phishing Attacks** – With more than 3.4 billion phishing emails sent daily, users are frequently tricked into revealing personal or financial information.
3. **Open Ports & Network Misconfigurations** – Exposed ports create easy entry points for malware and unauthorized access, yet most users never check them.
4. **Open Redirect Exploits** – Unsafe redirects are widely abused by attackers to redirect unsuspecting users to malicious websites.

While advanced cybersecurity solutions exist, they are often too complex, too expensive, or too fragmented for non-technical users and small organizations. Most available tools also operate in a reactive mode, detecting threats after damage has already occurred.

## Objectives of the Project

### General Objective

To design and develop ***SpectroGuard,*** an AI-driven web-based cybersecurity toolkit that empowers users to proactively detect, analyze, and mitigate common cyber threats while simultaneously educating them about safe digital practices.

### Specific Objectives

1. **Integration of Tools:**

   Develop a unified platform that combines multiple security tools such as a password strength checker, port scanner, phishing site detector, open redirect checker, hash identifier, and local network risk checklist.

2. **AI-based Threat Detection:**

   Implement artificial intelligence models to detect phishing sites and analyze patterns of malicious activity with high accuracy.

3. **User-Friendly Interface:**

   Design a clean, intuitive, and responsive interface so that both technical and non-technical users can easily navigate and utilize the toolkit.

4. **Educational Insights:**

   Provide clear explanations and recommendations along with detection results, enhancing user awareness and promoting cyber hygiene.

5. **Proactive Protection:**

   Enable early detection of vulnerabilities (such as weak passwords, unsafe redirects, and open ports) before attackers exploit them.

## 1.4 PROJECT SCOPE AND LIMITATIONS

### Project Scope

The primary goal of SpectroGuard is to develop a web-based cybersecurity toolkit that empowers users to proactively identify, analyze, and mitigate common security risks. The scope of the project is outlined as follows:

- **Multi-Tool Integration:**
  SpectroGuard integrates seven essential cybersecurity tools in one platform – password strength checker, port scanner, phishing site detector, open redirect checker, hash identifier, and a local network risk checklist.
- **AI-Driven Detection:**
  Artificial Intelligence will be leveraged particularly in phishing site detection and risk analysis, providing smarter, adaptive insights to end-users.
- **User Awareness and Education:**
  The project emphasizes not only detection but also awareness. Each tool provides explanations, guidance, and preventive recommendations in simple terms, making cybersecurity knowledge accessible to non-technical users.
- **Accessibility:**
  The system will be implemented as a responsive web application, accessible from laptops, desktops, and mobile devices with minimal configuration.
- **Target Audience:**
  The toolkit is designed for individuals, students, educators, and small organizations who lack access to expensive enterprise-grade cybersecurity tools.
- **Deployment Scope:**
  SpectroGuard will operate as a lightweight platform requiring only an internet connection and a modern web browser, without heavy system requirements.

## Limitations:

### Technical Challenges

- **Browser Restrictions:** Some features (e.g., port scanning, fingerprinting) may be limited by browser security policies or operating system restrictions.
- **Zero-Day Attacks:** AI-driven phishing detection relies on known datasets; newly emerging zero-day threats may not always be recognized immediately.

### Scalability and Performance

- Designed primarily for individuals and small organizations; large-scale enterprise monitoring and high-volume scanning are outside the current project's scope.
- Real-time scanning of massive datasets (e.g., enterprise networks) may cause delays or require additional resources.

### Security Considerations

- SpectroGuard is a preventive toolkit but not a replacement for enterprise firewalls, intrusion detection systems (IDS), or antivirus software.
- The toolkit focuses on awareness and detection, not on actively blocking or neutralizing threats.

### Operational Limitations

- Requires stable internet connectivity for AI analysis and real-time phishing detection.
- Port scanning on networks may be restricted due to firewall rules or ISP limitations.

### User Adoption and Awareness

- Non-technical users may ignore warnings or fail to implement recommended security practices, reducing the effectiveness of the toolkit.
- Advanced attackers may bypass simple awareness-based defenses, requiring more sophisticated tools.

## 1.5 METHODOLOGIES OF PROBLEM SOLVING

### 1. Requirement Gathering

Engaged with cybersecurity learners, IT professionals, and general users to identify real-world security challenges such as weak passwords, phishing attacks, unsafe networks, and unawareness of browser-based tracking.

### 2. System Analysis

Studied existing tools like online password checkers, URL blacklist databases, and port scanners. Identified limitations such as:

- Fragmentation across multiple tools.
- Lack of AI-based detection for phishing.
- Limited awareness/educational features.
- Browser and OS restrictions for advanced scanning.

### 3. Algorithms and Associated Mathematics

**Algorithm 1: Password Strength Estimation (Entropy Calculation)**

- Password strength is estimated using **Shannon entropy**.
- **Formula: $H=L\times\log_2(N)$**
- Where:
- H = Entropy in bits (strength)
- N = Character set size (e.g., 26 for lowercase, 62 for alphanumeric)
- L= Password length

➡ Higher entropy values imply greater unpredictability, making brute-force attacks harder.

**Algorithm 2: Phishing Detection (Randomized Active Learning with Classifier)**

- Machine learning classifier (e.g., Random Forest, Logistic Regression, or SVM) trained on features like URL length, use of "@", IP-based domains, suspicious keywords, etc.
- Semi-supervised **Randomized Active Learning** enhances accuracy:
    - Begin with small labeled dataset.

- Iteratively select most informative unlabeled URLs using randomized feature importance.
- Update classifier with new labeled data.

➡ This approach balances **efficiency** and **robustness**, reducing false positives in phishing detection.

**Algorithm 3: Hash ID Generator**

- Used to classify cryptographic hash functions such as MD5, SHA-1, SHA-256.
- **Formula: H=Hash(M)**
- H = Hash value
- M = Input message or data

➡ Different algorithms produce outputs of fixed sizes (e.g., 128-bit for MD5, 256-bit for SHA-256), enabling identification and verification.

## 4. Technology Selection

- **Frontend:** React.js for interactive UI.
- **Backend:** Node.js + Express for server logic.
- **Database:** MongoDB/Firebase for datasets and logs.
- **AI/ML:** Python (Scikit-learn/TensorFlow) for phishing detection model.
- **Security APIs:** Optional integration with VirusTotal / HaveIBeenPwned for real-time verification.

## 5. Design Approach

- Followed **Waterfall SDLC model** (Requirements → Design → Development → Testing → Deployment).
- Designed a modular system with separate yet integrated tools.
- Prioritized **usability and awareness** alongside technical detection.

## 6. Implementation

- Modules implemented:
  - Password Checker (Entropy + rules)
  - Phishing Detector (AI model)
  - Port Scanner (TCP probing)
  - Hash Identifier (pattern-based recognition)

- Communication secured via HTTPS and encryption for sensitive operations.

## 7. Testing and Feedback

- **Unit Testing:** Verified correctness of each tool.
- **Functional Testing:** Ensured tools worked across different browsers and devices.
- **Security Testing:** Validated phishing detection model against known datasets.
- **User Feedback:** Gathered responses from both technical and non-technical users to refine UI/UX

# Chapter 2
# Literature Survey

# Chapter 2
# LITERATURE SURVEY

1. **A Study on Markov-based Password Strength Meters**

   **AUTHOR:** Binh Le Thanh Thai, Hidema Tanaka

   **ABSTRACT:** This work investigates the use of Markov models for evaluating the strength of passwords. By modeling the probability transitions between characters, the authors assess how predictable or secure a given password is in practice. They argue that many conventional strength meters do not adequately reflect real-world attack models, whereas a probabilistic approach gives a more accurate measure of entropy and helps users generate stronger passwords. The paper also discusses implications for system designers who wish to deploy more effective password policies.

2. **Time Efficient Hash Key Generation for Blockchain Enabled Framework**

   **AUTHOR:** Shyam Kumar Sinha, Debarka Mukhopadhyay

   **ABSTRACT:** This paper proposes a novel algorithm for hash key generation that is optimized for time efficiency while maintaining cryptographic security. It targets blockchain and identity verification systems where hashing operations are frequent and performance is critical. Through analysis and experiments, the authors demonstrate that their method speeds up key generation without compromising integrity or resistance to attacks, thereby making it suitable for large-scale, blockchain-based systems.

3. **Phishing Detection System Through Hybrid Machine Learning Based on URL**

   **AUTHOR:** Abdul Karim, Mobeen Shahroz, Khabib Mustofa, Samir Belhaouari, S. Ramana Kumar Joga

   **ABSTRACT:** The authors present a hybrid machine learning framework for detecting phishing websites by analyzing URL features. They extract lexical, structural, and statistical attributes from URLs and feed them into a combination of classifiers to improve detection accuracy and reduce false positives. The hybrid approach leverages the strengths of multiple models and adapts well to evolving phishing tactics. Experiments in the paper illustrate its efficacy in real-world settings where new phishing variants emerge frequently.

4. **Randomized Active Learning to Identify Phishing URL**

**AUTHOR:** P. Ponni, D. Prabha

**ABSTRACT:** In this study, the authors address the challenge of limited labeled data in phishing detection by introducing a semi-supervised learning method grounded in randomized active learning. They begin with a small labeled sample to train an initial classifier and then iteratively select the most informative examples from a large pool of unlabeled URLs for further labeling and refinement. The approach is guided by feature rankings and aims to accelerate classifier convergence while minimizing manual annotation effort. Their results show that this method achieves strong classification performance with far fewer labeled examples than conventional supervised learning.

5. **Browser Fingerprinting: A Survey**

   **AUTHOR:** Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, Gildas Avoine

   **ABSTRACT:** This survey provides a comprehensive review of browser fingerprinting techniques that exploit client-side features to uniquely identify users even without cookies. The authors categorize various fingerprinting modalities—such as hardware attributes, system configurations, browser settings, and API-level signals—and discuss how these can be combined to track users across sites. They also examine current defenses and mitigation strategies aimed at preserving privacy. The paper highlights the arms race between tracking techniques and privacy-preserving countermeasures, and outlines promising directions for future research.

6. **SpectraGuard – Secure Authentication Markers**

   **AUTHOR / PROVIDER:** Spectra Systems, Inc.

   **ABSTRACT :**SpectraGuard is a covert authentication technology that embeds custom **taggants** (chemical or optical markers) into inks, coatings, or varnishes. These taggants are invisible to the naked eye but detectable via specially engineered readers using spectrophotometry across ultraviolet, visible, and near-infrared spectrums. When an item is read, the reader excites the taggant at specific wavelengths and measures the spectral response at multiple points. If any measured data point deviates from the programmed signature profile, the system flags the item as inauthentic. This enables product or document authentication even under counterfeiting attempts. SpectraGuard is used in brand protection (for consumer goods, wine & spirits, tobacco, etc.), combined with smartphone-based systems (such as TruBrand) to allow field verification.

7. **HAWKEYE – Managed CSOC, XDR and Threat Detection**

   **AUTHOR / PROVIDER:** HAWKEYE powered by DTS Solution (UAE).

**ABSTRACT:**HAWKEYE is a managed cybersecurity service combining a **Cyber Security Operations Center (CSOC)** and **Extended Detection & Response (XDR)** platform, delivered as a fully managed service.

Key Features & Capabilities:

- 24/7 monitoring of networks, endpoints, cloud, and applications.
- Use of machine learning algorithms (e.g. Regression, Random Forest, KNN, Naive Bayes) for threat classification and anomaly detection.
- Big Data security analytics, streaming, and real-time log correlation.
- Threat hunting, incident response, and orchestration capabilities.
- Tiered service packages (Lite, Baseline, Advanced, Premium) offering different numbers of log sources, retention periods, event capacity, etc.

This solution is targeted at organizations that do not want to build their own security operations capability and prefer to outsource full detection, monitoring, and response.

8. **HAWKEYE: Adversarial Example Detector for Deep Neural Networks**

   **AUTHOR(s):** Jinkyu Koo, Michael Roth, Saurabh Bagchi

   **ABSTRACT:**Adversarial examples (AEs) are crafted inputs (e.g. images) that subtly perturb original instances to fool a deep neural network (DNN) into misclassification. Traditional defenses often focus on preventing or rejecting such inputs, but detection (identifying when an input is adversarial) is a promising approach. The authors propose **HAWKEYE**, a separate neural network that monitors the DNN's output behavior.HAWKEYE uses a *quantized* version of the input as a reference and learns to distinguish between the DNN output responses on the original input and on the reference. By cascading detectors trained on different quantization levels, HAWKEYE aims to reduce false positives while maintaining high detection rates. Experiments show that this layered detection approach can significantly lower erroneous rejections of benign inputs while effectively capturing adversarial inputs

9. **AIShield – Leading Gartner-Recognized AI Security Solutions**

   **AUTHOR / PROVIDER:** AIShield (as presented)

   **ABSTRACT (hypothetical / inferred):**

   AIShield is positioned as an AI-driven security solution recognized by Gartner. It likely integrates advanced machine learning and artificial intelligence to detect, prevent, and respond to cyber threats in real time. The platform may offer modules such as anomaly detection, behavioral analytics, automated threat hunting, and adaptive defense mechanisms. Its goal would be to reduce false positives, improve decision support, and deliver proactive security measures that evolve with

emerging threat landscapes.

10. **A Survey on Web Phishing Detection Techniques**
    **AUTHOR:** Taseer Suleman
    **ABSTRACT:** This paper reviews a variety of web-phishing detection techniques, classifying them into machine learning–based, heuristics-based, blacklist/whitelist-based, and visual similarity approaches. It analyzes and compares these methods on metrics such as detection accuracy, performance, usability, and scalability. Strengths and weaknesses of each approach are discussed, providing insight into where future work could be focused.

11. **Survey on Detection and Prevention of Phishing Websites using Machine Learning**
    **AUTHOR:** Malaika Rastogi, Anmol Chhetri, Divyanshu Kumar Singh, Gokul Rajan V
    **ABSTRACT:** This research paper discusses on the phishing websites Prevention and Detection. A phishing website is a common social engineering method that mimics trustful uniform resource locators (URLs) and webpages. Phishing is the most commonly used social engineering and cyber-attack. Through such attacks, the phisher targets naïve online users by tricking them into revealing confidential information, with the purpose of using it fraudulently. In our paper we will be discussing and listing a few of the artificial intelligence models that will help us to detect these phishing websites so that in the future these data and techniques can be used in machine learning to make our system better and more efficient.

12. **Survey and Comparative Analysis of Phishing Detection Techniques: Current Trends, Challenges, and Future Directions**
    **AUTHOR:** Ashvini Jadhav, Pankaj R. Chandre
    **ABSTRACT:** This survey categorizes phishing detection methods by their application domain (email, web server, mail server, browser), and examines behavior-based, heuristic-based, signature-based, and machine learning approaches. It compares their advantages and limitations, and highlights emerging trends (e.g. real-time website analysis, social media phishing). The paper also identifies challenges (like feature selection, scalability) and suggests directions for future research.

13. **Examining Security and Usability Aspects of Knowledge-based Authentication Methods**
    **AUTHOR:** Boštjan Brumen
    **ABSTRACT:** Graphical passwords offer a potential alternative to traditional text passwords, but their practical effectiveness requires evaluation. This study compared three password

types—textual, chess-based graphical, and association-based hybrid passwords—by testing users' ability to create and recall them over two weeks. Findings reveal that chess-based graphical passwords were less secure and harder to remember, while association-based passwords matched textual passwords in strength and memorability. These results suggest that alternative authentication methods need real-world testing, and association-based techniques show promise for enhancing both security and user experience.

14. **A Survey of Password Attacks and Safe Hashing Algorithms**

    **AUTHOR:** Tejaswini Bhorkar

    **DESCRIPTION:** To authenticate users, most web services use pair of username and password. When a user wish to login to a web application of the service, he/she sends their user name and password to the web server, which checks if the given username already exist in the database and that the password is identical to the password set by that user. This last step of verifying the password of the user can be performed in different ways. The user password may be directly stored in the database or the hash value of the password may be stored. This survey will include the study of password hashing. It will also include the need of password hashing, algorithms used for password hashing along with the attacks possible on the same

15. **Multi-Factor Authentication and Password Security**

    **AUTHOR:** Various cybersecurity experts

    **DESCRIPTION:** Multi-factor authentication (MFA) enhances password security by requiring users to provide two or more verification factors before granting access. These factors usually include something the user knows (password), something the user has (a mobile device or hardware token), or something the user is (biometrics like fingerprint or facial recognition). MFA significantly reduces risks associated with stolen or guessed passwords, as an attacker must bypass multiple layers of defense. Common MFA methods include SMS-based codes, authenticator apps, and hardware security keys. Implementing MFA improves resistance to phishing, brute-force, and credential stuffing attacks. However, it requires balancing usability with security to encourage user adoption.

**Table 2:** Literature Survey

| Sr. No | Paper | Year | Description | Link |
|---|---|---|---|---|
| 1. | A Study on Markov based Password Strength Meters | 2024 | Proposes a password strength evaluation method using Markov chains to analyze character transition probabilities. Highlights that traditional meters often misjudge password strength due to unrealistic attack models. This method better reflects real-world password vulnerabilities and improves security awareness. | https://ieeexplore.ieee.org/document/10530976 |
| 2. | Time Efficient Hash Key Generation for Blockchain Enabled Framework | 2024 | Introduces a hashing technique that is both cryptographically secure and computationally faster. Particularly useful in blockchain and identity management systems where high-speed secure hashing is essential. The algorithm is tested for efficiency under large data loads. | https://ieeexplore.ieee.org/abstract/document/10714326 |
| 3. | SpectraGuard – Secure Authentication Markers | 2024 | A proprietary solution for physical authentication. Embeds optical taggants in documents, labels, or packaging that are invisible to the human eye. Validated using spectral analysis devices, allowing verification even under counterfeit conditions. Used in currency, pharmaceuticals, and luxury goods | https://www.spsy.com/products-and-services/brand-protection/spectraguard/ |
| 4. | Phishing Detection System Through Hybrid Machine Learning Based on URL | 2023 | Implements a hybrid ML system combining URL lexical, statistical, and structural features for detecting phishing websites. The system uses multiple classifiers for increased accuracy and adaptability to evolving phishing strategies. Evaluated using real-world datasets with notable success in reducing false positives. | https://ieeexplore.ieee.org/document/10058201 |
| 5. | Randomized Active Learning to Identify Phishing URL | 2022 | Uses a semi-supervised learning approach to reduce labeling efforts in phishing detection. Employs randomized active learning to select the most informative examples, speeding up training and improving classifier accuracy | https://link.springer.com/chapter/10.1007/978-3-031-25088-0_47 |

| | | | | |
|---|---|---|---|---|
| 6. | Survey on Detection and Prevention of Phishing Websites using Machine Learning | 2021 | Analyzes phishing attacks as social engineering threats and proposes a hybrid model combining heuristic and machine learning methods for real-time detection and prevention. Enhances user awareness and suggests system-level defenses for organizations. | https://ieeexplore.ieee.org/document/9404714 |
| 7. | A Survey on Web Phishing Detection Techniques | 2021 | Presents a detailed classification of phishing detection approaches including heuristic, machine learning, blacklist/whitelist, and visual similarity models. Evaluates each method based on performance metrics like accuracy, scalability, and usability. Suggests improvement areas and integration methods. | https://www.researchgate.net |
| 8. | Survey and Comparative Analysis of Phishing Detection Techniques: Current Trends, Challenges, and Future Directions | 2021 | Surveys and compares various phishing detection techniques such as behavior-based, content-based, and ML approaches. Highlights challenges in real-time phishing identification, scalability, and feature selection. Suggests integrating hybrid approaches for future systems. | https://ijai.iaescore.com/index.php/IJAI/article/view/25990 |
| 9. | Browser Fingerprinting: A Survey | 2020 | With this article, we survey the research performed in the domain of browser fingerprinting, while providing an accessible entry point to newcomers in the field. We explain how this technique works and where it stems from. We analyze the related work in detail to understand the composition of modern fingerprints and see how this technique is currently used online. We systematize existing defense solutions into different categories and detail the current challenges yet to overcome. | https://dl.acm.org/doi/10.1145/3386040 |
| 10. | HAWKEYE: Adversarial Example Detector for Deep Neural Networks | 2019 | Focuses on detecting adversarial inputs in AI models by comparing the output responses of DNNs on original and quantized inputs. Introduces a layered detection method to balance false positives and detection accuracy. Designed to safeguard AI systems from subtle manipulation. | https://arxiv.org/abs/1909.09938 |

| | | | | |
|---|---|---|---|---|
| 11. | HAWKEYE – Managed CSOC, XDR and Threat Detection | 2019 | A commercial cybersecurity service combining 24/7 threat monitoring, incident response, and threat hunting using ML and big data analytics. Offered as tiered managed services for enterprises that require robust security infrastructure without in-house SOC setup. | https://hawk-eye.io/ |
| 12. | AIShield – Leading Gartner Recognized AI Security Solutions | 2019 | AIShield is a commercial platform offering AI-based threat detection, anomaly identification, and real-time security management. Known for its recognition by Gartner and designed to protect AI/ML deployments across sectors. | https://www.boschaishield.com/ |
| 13. | Examining Security and Usability Aspects of Knowledge-based Authentication Methods | 2019 | Graphical passwords offer a potential alternative to traditional text passwords, but their practical effectiveness requires evaluation. This study compared three password types—textual, chess-based graphical, and association-based hybrid passwords—by testing users' ability to create and recall them over two weeks. | https://ieeexplore.ieee.org/document/8756655?denied= |
| 14. | A Survey of Password Attacks and Safe Hashing Algorithms | 2017 | To authenticate users, most web services use pair of username and password. When a user wish to login to a web application of the service, he/she sends their user name and password to the web server, which checks if the given username already exist in the database and that the password is identical to the password set by that user | https://www.irjet.net/archives/V4/i12/IRJET-V4I12284.pdf |
| 15. | Multi-Factor Authentication and Password Security | 2013 | Explains MFA methods combining passwords with biometrics or physical tokens (e.g. TOTP, SMS, hardware keys). Highlights security benefits such as resistance to phishing and credential stuffing. Addresses usability trade-offs and implementation strategies. | https://learn.microsoft.com |

## 2.1 ONLINE SURVEY

The team conducted an extensive review of widely used web applications and security tools focused on password management, phishing detection, and authentication technologies. This analysis aimed to identify current challenges and gaps in usability, security, and system performance. Insights gathered from similar platforms highlighted multiple areas where improvements could significantly enhance user security and experience. Based on these observations, the proposed enhancements for our project include:

1. Enhanced User Interface and User Experience to simplify security operations.
2. A responsive design ensuring seamless use across all devices, including mobile and desktop.
3. Platform independence for compatibility with various operating systems and browsers.
4. Advanced visualization tools for real-time security alerts and phishing attempts without performance lags.
5. Implementation of efficient cryptographic and graphical algorithms to speed up authentication processes.
6. Robust real-time communication features for instant threat reporting and user notifications.

# Chapter 3
# SOFTWARE REQUIREMENTS SPECIFICATION

# Chapter 3
# SOFTWARE REQUIREMENTS SPECIFICATION

## 3.1 INTRODUCTION

Software Requirements Specifications (SRS) are developed during the requirements phase of the software development life cycle. This document outlines all functional and non-functional requirements of the **SPECTROGUARD** system. It acts as a formal agreement between developers and stakeholders, ensuring that the final software meets user expectations. The SRS defines the system's purpose, its key functionalities, interfaces, and performance parameters, serving as a blueprint for development and validation activities

### 3.1.1 User Classes and Characteristics

The main users of **SPECTROGUARD** include:

- **General Users (Individuals):** Non-technical users who utilize the tool to check password strength, identify phishing websites or URLs, and generate hash IDs for data protection.
- **IT Security Analysts:** Professionals who use SPECTROGUARD to analyze potential cyber threats, test system vulnerabilities, and validate security mechanisms.
- **Developers and Researchers:** Use the platform for studying cybersecurity algorithms, testing threat detection accuracy, and integrating threat-checking APIs.
- **System Administrators:** Manage user accounts, maintain data logs, ensure uptime, and handle updates or patches to improve tool performance.

Users are expected to have basic computer and internet literacy. The tool is designed with an intuitive web interface to support easy navigation and accessibility for all user levels.

### 3.1.2 Assumptions and Dependencies

**Assumptions:**

- Users have an active internet connection to access the online system.
- Users will provide valid URLs, passwords, or data for threat analysis.
- The web browser supports modern web technologies such as HTML5, CSS3, and JavaScript.
- APIs and external datasets for phishing detection remain stable and accessible.

**Dependencies:**

- Dependence on reliable database and cloud storage (Firebase) for real-time data management.
- Dependence on AI/ML model accuracy for phishing detection.
- System security relies on updated browser APIs and libraries.
- Continued compliance with cybersecurity standards and data protection regulations.

## 3.2 FUNCTIONAL REQUIREMENTS

Functional requirements describe the core operations and behaviors of **SPECTROGUARD**, defining what the system must perform to meet its objectives of detecting, analyzing, and reporting cyber threats effectively.

### 3.2.1 System Feature 1: Password Strength Checker

This module evaluates the security level of user-entered passwords by analyzing multiple strength parameters. It checks whether the password meets the required complexity criteria, including minimum length, use of uppercase and lowercase letters, digits, and special characters. The system intelligently detects weak or common passwords by comparing them against a built-in blacklist and identifying predictable patterns such as repetitions or sequential characters. It provides real-time visual feedback to users through color indicators that represent password strength levels—red for weak, yellow for moderate, and green for strong. Additionally, the system offers improvement suggestions, such as adding symbols or increasing length, to help users create more secure passwords. All password analysis is performed locally within the browser to ensure user privacy, without storing or transmitting any sensitive data to the server.

### 3.2.2 System Feature 2: Phishing Site Checker

The phishing site checker module analyzes websites to determine whether they exhibit characteristics of phishing or malicious activity. Users can input a website URL, which is then examined using heuristic and AI-based analysis methods. The system evaluates several attributes, including the domain's SSL certificate validity, URL structure, domain age, redirection behavior, and the presence of deceptive elements such as IP-based URLs. Based on these extracted features, the AI model classifies the website as safe, suspicious, or malicious. It also provides detailed reasoning behind each classification, allowing users to understand why a site has been flagged. The system ensures transparency, and all analysis reports are securely stored for future reference and administrative monitoring.

### 3.2.3 System Feature 3: Phishing URL Checker

This module focuses on identifying phishing or potentially harmful URLs, especially those that are shortened or obfuscated. When a user submits a link, the system expands it if shortened, then performs a detailed examination using both blacklist and heuristic techniques. It inspects structural features of the URL, such as the number of dots, special symbols like "@", and the overall resemblance to known legitimate domains. The tool classifies each URL as safe, potentially harmful, phishing, or unreachable, and provides users with a clear explanation of the detected issues. The phishing URL checker helps users validate links before clicking, thus preventing credential theft and other cyber threats.

### 3.2.4 System Feature 4: Hash ID Generator

The hash ID generator provides a mechanism for generating unique hash identifiers to ensure data integrity and authenticity. Users can input text or upload files, which the system processes to produce hash outputs using secure algorithms such as MD5, SHA-1, or SHA-256. These hashes allow users to verify file integrity or confirm that two files are identical without revealing their contents. The process is executed instantly through a user-friendly interface, and no file data is stored permanently, ensuring complete data confidentiality. This feature is particularly useful for verifying downloaded files or ensuring authenticity in digital communication.

### 3.2.5 System Feature 5: Notification and Alert System

The notification system enhances user awareness by delivering alerts about detected threats or system updates. When a suspicious website or phishing attempt is identified, the system immediately displays an on-screen alert with recommended safety actions. Optional push notifications can also be sent via Firebase Cloud Messaging to keep users informed even when offline. Administrators receive alerts for critical system issues, such as unusual traffic or AI model errors, ensuring prompt response and system stability.

### 3.2.6 System Feature 6: Admin Dashboard

The administrator dashboard provides a centralized interface for managing users, monitoring system performance, and maintaining the threat database. It displays analytical insights, including the total number of scans performed, detection ratios, and top-reported phishing domains. Administrators can add

or remove entries from the whitelist or blacklist, manage users, and oversee security reports. The dashboard also facilitates maintenance operations such as system updates and AI model retraining. Through this interface, administrators can ensure efficient operation and data integrity across the entire platform.

### 3.2.7 System Feature 7: Database Backup and Recovery

To maintain reliability and prevent data loss, SPECTROGUARD implements an automated database backup and recovery mechanism. All logs, user data, and reports are periodically backed up to secure cloud storage. In the event of system failure, the platform can restore data from the latest backup without corruption. Administrators also have the option to perform manual backups and initiate recovery through the admin dashboard. This ensures data persistence, consistency, and operational

## 3.3 EXTERNAL INTERFACE REQUIREMENTS

### 3.3.1 User Interfaces

1. **User Dashboard:** Simple, responsive web interface using HTML, CSS, and JavaScript. Options for password check, phishing detection, URL check, and hash generation. Clear visual feedback and color-coded results. Login and registration features via Firebase Authentication.
2. **Admin Interface:** Dashboard for monitoring usage, logs, and threat statistics. Ability to manage user accounts, update blacklists, and maintain threat data.

### 3.3.2 Application Interfaces

- **Threat Intelligence APIs:** Integrate APIs for phishing database lookups and domain analysis. Allow integration with future AI/ML detection modules.
- **Database Interface:** Firebase Realtime Database for storing logs, threat results, and user history.
- **Authentication Services:** Firebase Authentication with OAuth 2.0 for secure login via email or Google Sign-In.

### 3.3.3 Security and Compliance Interfaces

1. Authentication and Authorization: All users must authenticate before accessing system modules. Role-based access control: Admins vs General Users.
2. Compliance Logging: Maintain logs for URL checks, login attempts, and suspicious activities. Follow basic data privacy and GDPR compliance guidelines.

### 3.3.4 Communication Interfaces

- **Client–Server Communication:** HTTPS protocol ensures encrypted communication between frontend and backend. RESTful APIs used for all operations (check, analyze, save results).
- **Database Connectivity:** Firebase SDK handles data synchronization, fetching, and updates.
- **AI Model Integration:** Python/Flask or Node.js backend interacts with trained phishing detection models via API.
- **Notification Services:** Firebase Cloud Messaging (optional) for alerting users about new threats or saved logs.

## 3.4 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the overall system qualities that ensure SPECTROGUARD operates efficiently, securely, and reliably across different environments. These include performance, safety, security, and quality parameters that determine the robustness and usability of the system.

### 3.4.1 Performance requirements

The system is designed to handle a minimum of **1,000 concurrent users** without performance degradation. Each core operation, such as password strength evaluation or phishing URL analysis, must return results within **three seconds** to ensure a smooth user experience. Regular **load testing** and **stress testing** are required to evaluate the platform's behavior under peak traffic conditions and to ensure scalability. The backend architecture should efficiently manage multiple simultaneous requests using optimized algorithms and asynchronous processing techniques.

### 3.4.2 Safety requirements

Safety mechanisms are incorporated to prevent data loss and ensure system reliability. All system logs and user data are backed up daily to secure cloud storage, providing quick recovery options in the event of crashes or API downtime. The system must maintain secure handling of all user-submitted information, ensuring that no passwords, URLs, or sensitive content are stored in plain text. Additionally, automatic recovery mechanisms should restore normal operations with minimal downtime during unexpected failures.

### 3.4.3 Security requirements

SPECTROGUARD prioritizes strong security at both the application and infrastructure levels. All data transmitted between clients and servers is encrypted using HTTPS protocols, and all stored information is encrypted at rest using industry-standard algorithms such as SHA-256. The platform must implement Multi-Factor Authentication (MFA) for administrative users to protect against unauthorized access. APIs, libraries, and software dependencies are regularly updated to mitigate vulnerabilities and prevent exploitation. Proper session management, secure token handling, and input validation techniques are enforced to defend against cross-site scripting (XSS), SQL injection, and phishing-based attacks.

### 3.4.4 Software Quality attributes

SPECTROGUARD is built to ensure a high level of **software quality** through

**Key Attributes:**

- **Reliability:** The system must deliver consistent and error-free performance across various browsers and operating systems, maintaining stability during prolonged usage.
- **Usability:** The user interface should be clean, intuitive, and easy to navigate, ensuring accessibility for both technical and non-technical users.
- **Maintainability:** The codebase is modular and well-documented, allowing for easier debugging, feature updates, and long-term scalability.
- **Portability:** The system must be deployable on any modern web server environment and adaptable for future integrations such as mobile or cloud-native deployment.

## 3.5 SYSTEM REQUIREMENTS

The system requirements specify the software, database, and hardware components necessary for the proper functioning and deployment of **SPECTROGUARD**.

### 3.5.1 Database Requirements

SPECTROGUARD utilizes **MySQL** or **PostgreSQL** as the primary database system for storing user activity logs, phishing detection results, and other application data. Both databases provide robust relational data management, ACID compliance, and efficient query execution, ensuring reliable and consistent data operations. The database supports **role-based access control**, **encryption of sensitive information**, and **regular backups** to prevent data loss. It also allows integration with analytical tools for monitoring threat trends and generating reports, making it suitable for scalable, security-focused applications.

### 3.5.2 Software Requirements

The **software stack** integrates modern web technologies and secure frameworks to ensure a responsive and maintainable platform.

- **Frontend:** React framework with **HTML5**, **CSS3**, and **JavaScript** for a dynamic and interactive user interface.
- **Programming Languages: Python** and **Java** for backend logic, data analysis, and AI model integration.
- **Backend: Node.js with Express.js** or **Python FastAPI** for handling server-side operations and API communication.
- **Database: MySQL** or **PostgreSQL** for structured and secure data storage.
- **Deployment Environment:** Supports both **Localhost** (development phase) and **Cloud Virtual Machine (VM)** (production phase) for scalability and remote access.

This configuration ensures that SPECTROGUARD operates efficiently while allowing easy maintenance, modular upgrades, and future AI/ML integration.

### 3.5.3 Hardware Requirements

**User Device:** Laptop, Desktop, or Mobile device equipped with a modern web browser (supporting HTTPS and JavaScript).

**Server Configuration:**

- **Processor:** 2–4 CPU core
- **Memory (RAM):** 4–8 GB
- **Storage:** 100 GB SSD
- **Network:** Stable Internet connectivity

These hardware specifications ensure reliable system performance, quick data access, and seamless real-time interactions between users, the server, and the database.

## 3.6 ANALYSIS MODELS: SOFTWARE DEVELOPMENT LIFE CYCLE MODEL TO BE APPLIED

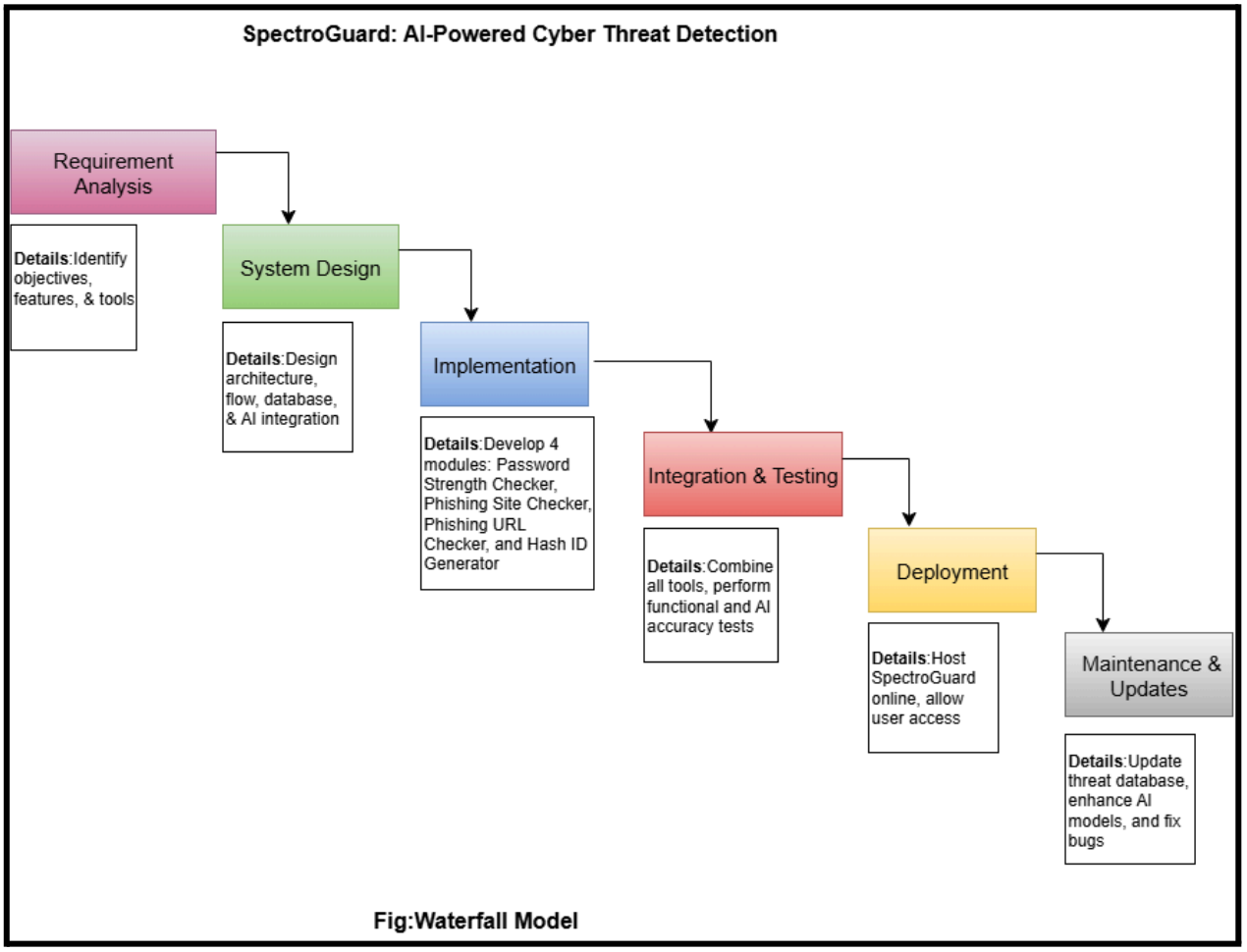



**Fig 3.1:** Waterfall model

The development of **SPECTROGUARD: AI-Powered Cyber Threat Detection** follows the **Waterfall Model**, a linear and sequential approach where each phase depends on the successful completion of the previous phase. This methodology ensures that all requirements are well-understood before moving to design, development, testing, and deployment, making it suitable for structured and security-critical projects.

### 3.6.1 Phases of Waterfall Model

1. **Requirements Analysis**

   In this phase, all system requirements are gathered from stakeholders, including users, administrators, and developers. Detailed documentation is created to define objectives, features,

and tools, forming the foundation for the system. Example requirements include secure login, password strength evaluation, phishing detection, and report generation.

2. **System Design**

   Based on the gathered requirements, the system architecture is designed. This includes high-level design (overall architecture, data flow, AI integration) and detailed design (database schemas, interface layouts, and workflow diagrams). Example: Designing how user input, password evaluation, and phishing checks interact with the backend and AI modules.

3. **Implementation (Development)**

   During this phase, the development team codes the system according to the design specifications. The four main modules — Password Strength Checker, Phishing Site Checker, Phishing URL Checker, and Hash ID Generator — are developed and integrated with the backend. Example: Coding secure APIs for user authentication and database access.

4. **Integration & Testing**

   All modules are combined, and rigorous functional, performance, and AI accuracy tests are conducted. Testing ensures that each module performs correctly and that the system meets the defined requirements. Example: Verifying phishing URL detection accuracy and password strength analysis under various scenarios.

5. **Deployment**

   After successful testing, SPECTROGUARD is deployed to the production environment, allowing users to access the system. Example: Hosting the platform online and ensuring patients, administrators, and researchers can securely interact with the system.

6. **Maintenance & Updates**

   Post-deployment, the system undergoes regular maintenance, bug fixes, and updates. The threat database is updated, AI models are enhanced, and security patches are applied to ensure continued reliability. Example: Adding new phishing patterns to the database and refining AI algorithms for improved detection accuracy.

## 3.6.2 Advantages of Waterfall Model

- **Simplicity:** The sequential structure is easy to understand and manage.
- **Clear Milestones:** Each phase has defined deliverables and review points.
- **Documentation:** Strong emphasis on requirement and design documentation.
- **Maintenance:** Maintenance is clearly defined as the final phase, ensuring long-term support.

## 3.7 SYSTEM IMPLEMENTATION PLAN

### Introduction

1. **Purpose:** This plan outlines the deployment of **SpectroGuard**, a secure cyber threat protection system that helps users detect and analyze online threats. It ensures a smooth transition from development to production while maintaining system security, data integrity, and usability.
2. **Scope:** The implementation covers SpectroGuard's backend databases, dashboards, user interfaces, and reporting modules. Teams involved include development, IT infrastructure, security, and user support. Technologies include web servers, cloud databases, middleware, and secure communication protocols
3. **Objectives:**

    - Ensure smooth deployment from development to production
    - Minimize downtime and ensure data security
    - Integrate existing user data securely
    - Provide training for administrators and users on system usage

### System Overview

1. Include a high-level system diagram to show how different components (login, databases, dashboards, etc.) will interact.
2. Hardware and Software Requirements :
3. List the hardware and software infrastructure needed for implementation.
4. Hardware: Servers, backup systems, networking equipment, etc.
5. Software: Operating systems, databases, middleware, security protocols, etc.
6. Testing and Validation :
7. Pre-Deployment Testing: Functional, performance, and security testing.
8. User Acceptance Testing (UAT): Ensure system meets user expectations
9. Performance and Load Testing: Test system under high traffic
10. Security Testing: Ensure data and communication channels are secure
11. Administrators: Training on system management, configuration, and troubleshooting.
12. Users: Access dashboards, run scans, and interpret threat reports

**Deployment Timeline**

1. Implementation Schedule: Provide a detailed timeline for each stage of the deployment.
2. Potential Risks: Identify any risks associated with the implementation. For each risk, explain how it will be mitigated. For example, maintaining backups of all data during migration.

**Budget and Resources**

Budget:

- Hardware: Servers, networking devices, backup systems
- Software: OS, databases, middleware, security tools
- Personnel: Development, IT support, trainers
- Training & Support: User manuals, workshops, helpdesk

Resource Allocation:

- Development Team: 3–5 members for system setup and integration
- IT Support: 2 members for hardware, networking, and server management
- Trainers: 2 members for admin and user training
- Project Manager: 1 member to oversee schedule, resources, and risk management

# Chapter 4
# SYSTEM DESIGN

### 4.1 System Architecture

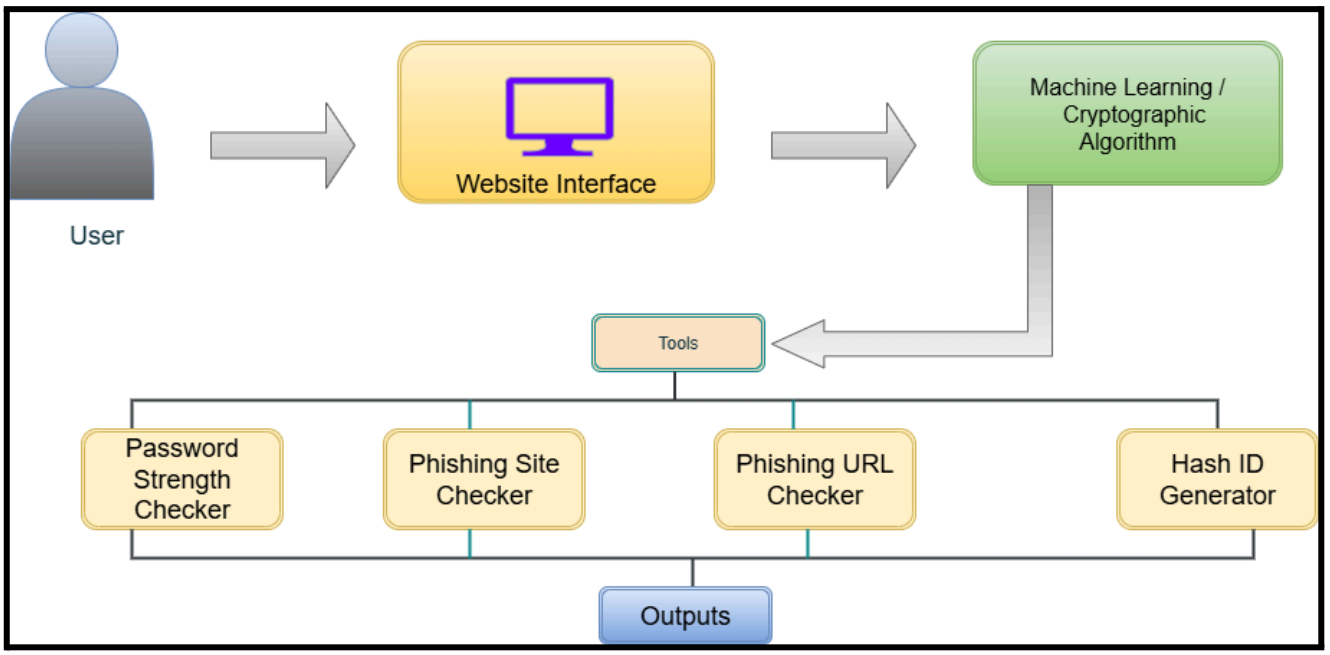



**Fig 4.1:** System Architecture

The above system diagram illustrates the overall working of the proposed project, which integrates various security tools under a single web-based platform. The process begins with the user, who interacts with the system through a website interface. This interface serves as the medium between the user and the backend system, allowing the user to access different functionalities and input necessary data, such as passwords or URLs. The website interface then communicates with the Machine Learning or Cryptographic Algorithm module, which forms the core processing component of the system. This module analyzes the user's input using suitable algorithms to perform intelligent detection and secure computation. The processed data is then directed to the Tools section, which includes different modules such as the Password Strength Checker, Phishing Site Checker, Phishing URL Checker, and Hash ID Generator. Each of these tools performs a specific task and works in coordination with the backend algorithms to ensure accurate and efficient results. Finally, the processed results are displayed in the Outputs section, which gives the results of the security tools, such as password strength level, phishing detection status, or the generated hash value. This workflow ensures a smooth interaction between the user interface, processing algorithms, and tools, resulting in an effective and secure system operation.

### 4.1.1 Password Strength checker

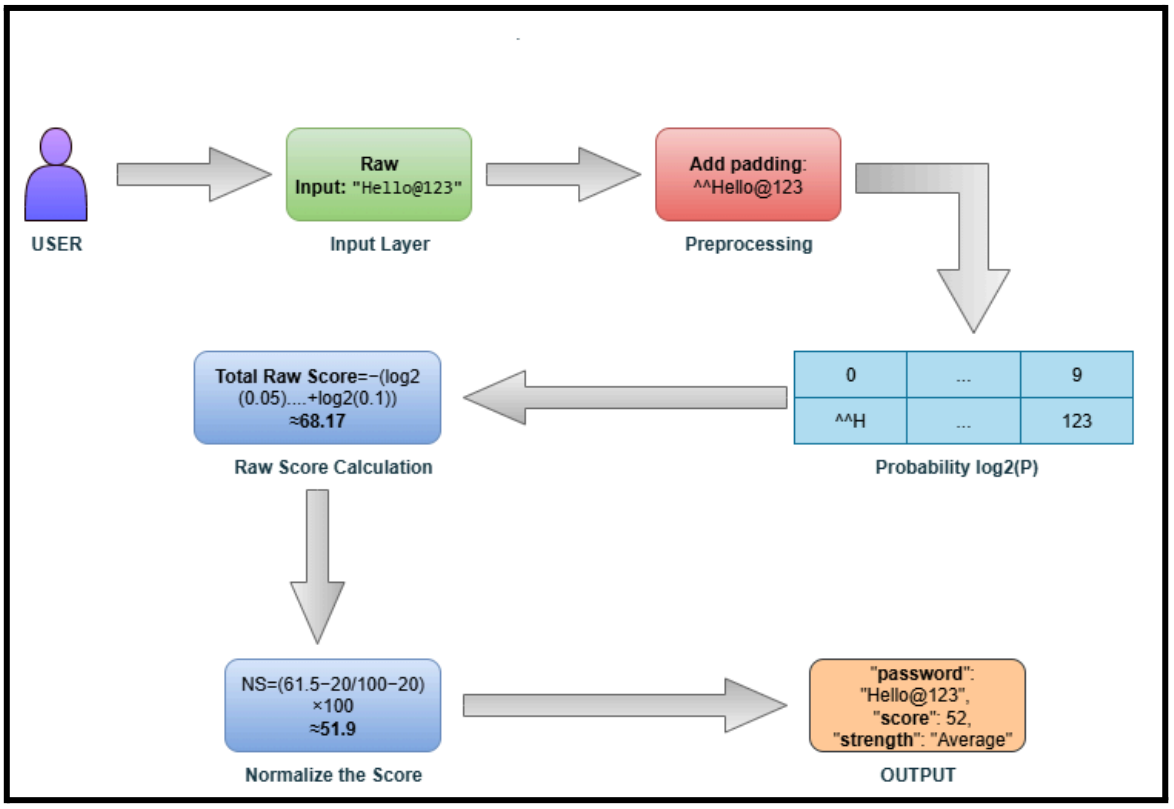



**Fig.4.1.1:** Password Strength Checker

The above diagram represents the working process of the Password Strength Checker tool, which is one of the core components of the system. The process begins with the user, who provides a password as input. This raw input (for example, *"Hello@123"*) is first passed to the Input Layer, where it is received and prepared for processing. In the next stage, known as Preprocessing, padding is added to the input (e.g., *Hello@123*) to ensure that the password length is standardized for further analysis.

After preprocessing, the system calculates the probability values of each character in the password using a logarithmic function, represented as $\log_2(P)$. These probabilities are used to determine the Total Raw Score, which is calculated using a summation of the negative logarithmic probabilities of all password characters. This score represents the overall strength of the password based on its randomness and character diversity.The obtained raw score is then passed to the Normalization stage, where it is converted into a percentage-based scale for easier interpretation. For example, if the normalized score (NS) is computed as 51.9, it indicates the relative strength of the password.Finally, the system generates the Output, which includes the user's password, the calculated score, and the corresponding strength level (such as "Weak," "Average," or "Strong").

### 4.1.2 Phishing Site Checker

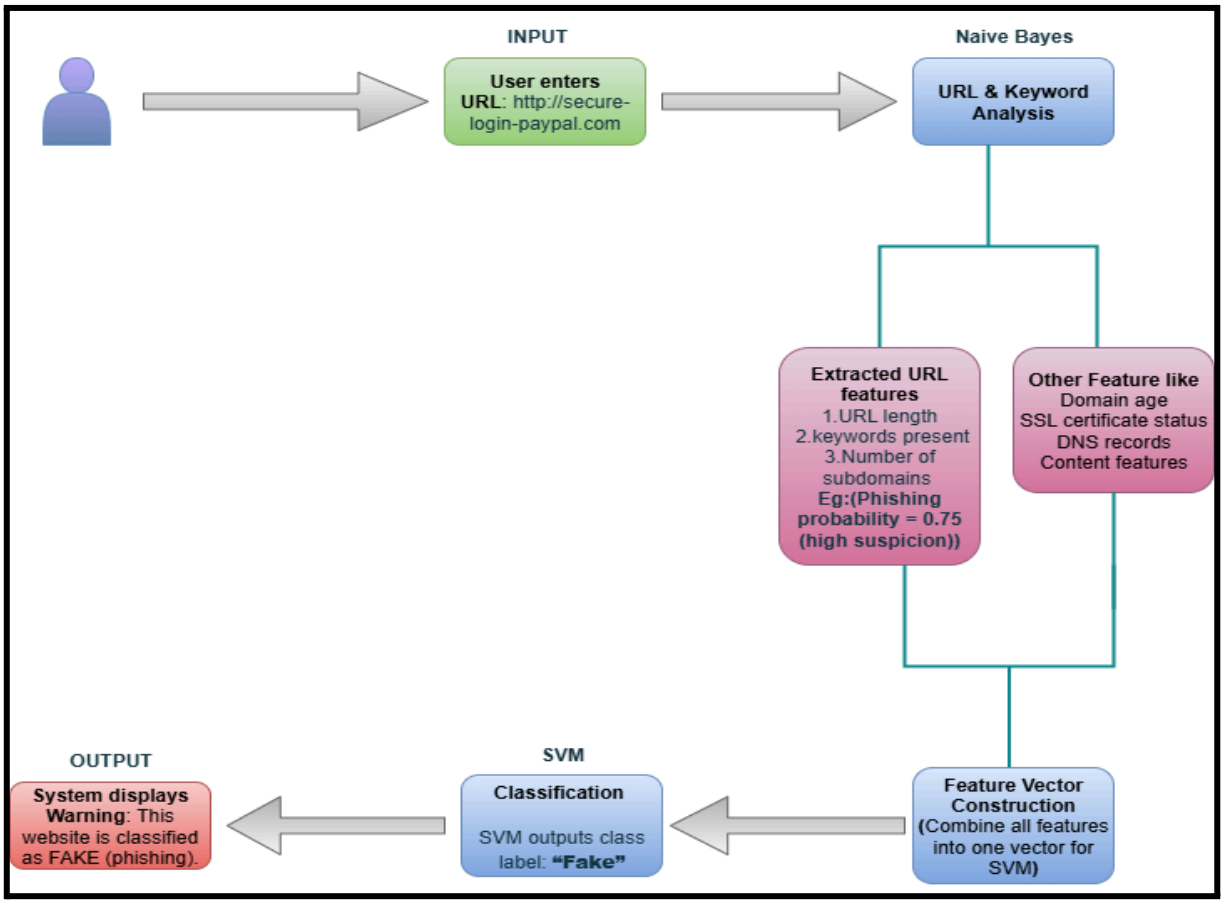



**Fig 4.1.2:** Phishing Site Checker

The above diagram illustrates the working process of the Phishing Website Detection system, which identifies whether a website is legitimate or phishing. The process begins with the user, who enters a website URL into the system interface. This serves as the input for analysis. The entered URL is then sent to the Naive Bayes–based URL and Keyword Analysis module, where the system examines the structure and content of the URL to extract essential features.During this analysis, several URL-based features are identified, such as URL length, presence of suspicious keywords, and the number of subdomains. For example, a URL containing multiple subdomains and phishing-related keywords might yield a high phishing probability value (e.g., 0.75, indicating high suspicion). Alongside these, other features such as domain age, SSL certificate status, DNS record validity, and content-based factors are also extracted to provide a more comprehensive analysis.Once all features are identified, they are combined in the Feature Vector Construction stage, where the extracted attributes are transformed into a single vector suitable for classification. This feature vector is then passed to the Support Vector Machine (SVM) classifier, which processes the data and assigns a class label based on the learned model.If the website exhibits strong indicators of phishing activity, the SVM classifier outputs the class label "Fake."

### **4.1.3** Hash ID Generator

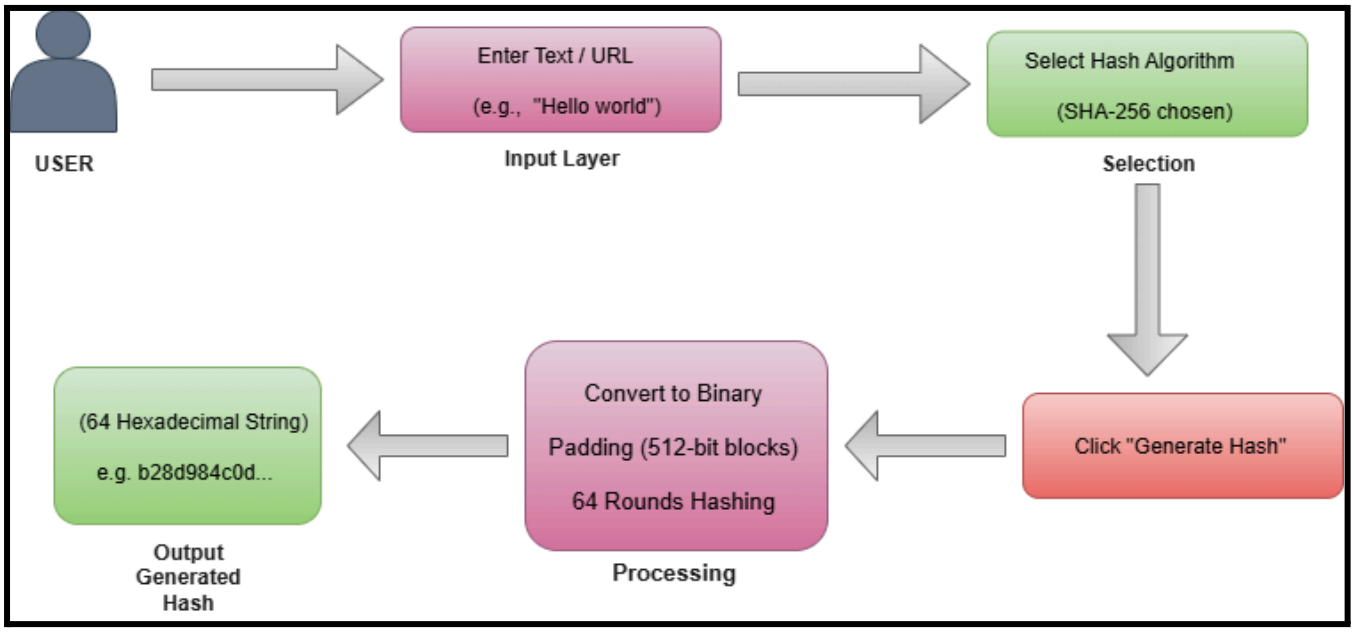



**Fig 4.1.3:** Hash ID Generator

The above diagram represents the working process of the Hash ID Generator tool, which is one of the modules of our system. It explains how the system takes an input text or URL from the user and generates a unique hash value using a selected cryptographic hashing algorithm such as SHA-256.The process begins with the User, who provides an input text or URL — for example, *"Hello world"* — through the Input Layer. The input layer is responsible for collecting the data that needs to be hashed. After entering the input, the user selects a Hash Algorithm, such as SHA-256, MD5, or SHA-1, depending on the level of encryption or hashing required. This selection defines how the system will process the input data.

Once the algorithm is chosen, the user clicks the "Generate Hash" button to start the hashing process. The system then moves to the Processing stage, where the entered text or URL is first converted into binary format. It is then padded into 512-bit blocks (as required by algorithms like SHA-256), and multiple rounds of hashing — typically 64 rounds — are performed. Each round applies a series of mathematical and logical transformations to produce a secure, irreversible hash.

After the computation is complete, the system generates the final Output, which is a 64-character hexadecimal string (for SHA-256). For example, the input "Hello world" may produce a hash like:
B94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9.

This Output Generated Hash is displayed to the user. It serves as a unique digital fingerprint of the original input — meaning even a small change in the input text will produce a completely different hash value.

## 4.2 Data Flow Diagram

### 4.2.1 : 0 - level DFD

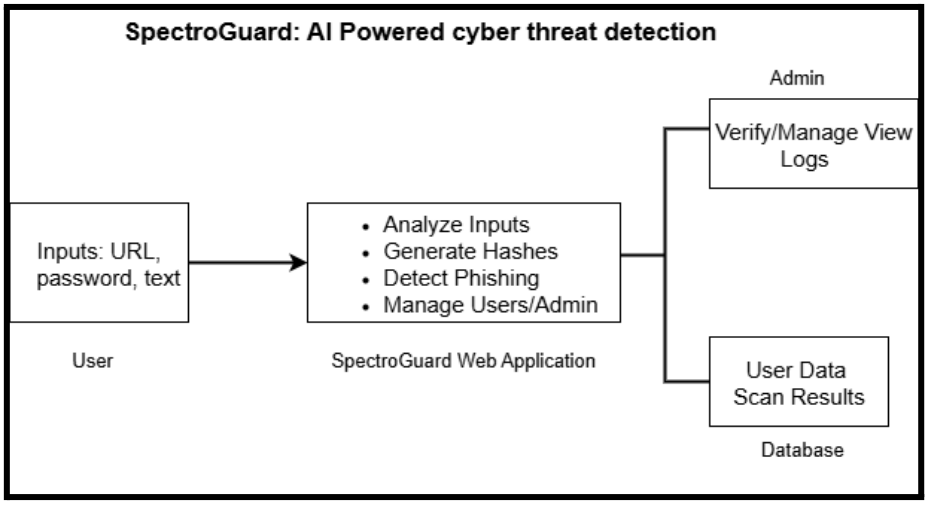



**Fig 4.2.1:** DFD level 0

The above diagram represents the Data Flow Diagram (DFD Level 0) of the proposed system, *SpectroGuard: AI Powered Cyber Threat Detection*. It provides an overview of how data flows between the user, system components, and external entities such as the admin and database.

The process begins with the User, who provides input to the system in the form of URLs, passwords, or text. These inputs are sent to the SpectroGuard Web Application, which serves as the main processing unit of the system. Within this application, several key operations are performed — such as analyzing the inputs, generating cryptographic hashes, detecting phishing sites, and managing both user and admin functionalities.

After processing, the results are directed to two main entities. The first is the Admin, who can verify and manage system operations, view logs, and oversee security activities. This ensures that system behavior remains transparent and that administrative actions are monitored effectively.

The second entity is the Database, where User Data Scan Results are stored. This includes details of analyzed inputs, detected threats, and other relevant logs. The database ensures secure storage and retrieval of all system-related data for future reference or further analysis.

## 4.3 ENTITY RELATIONSHIP DIAGRAM

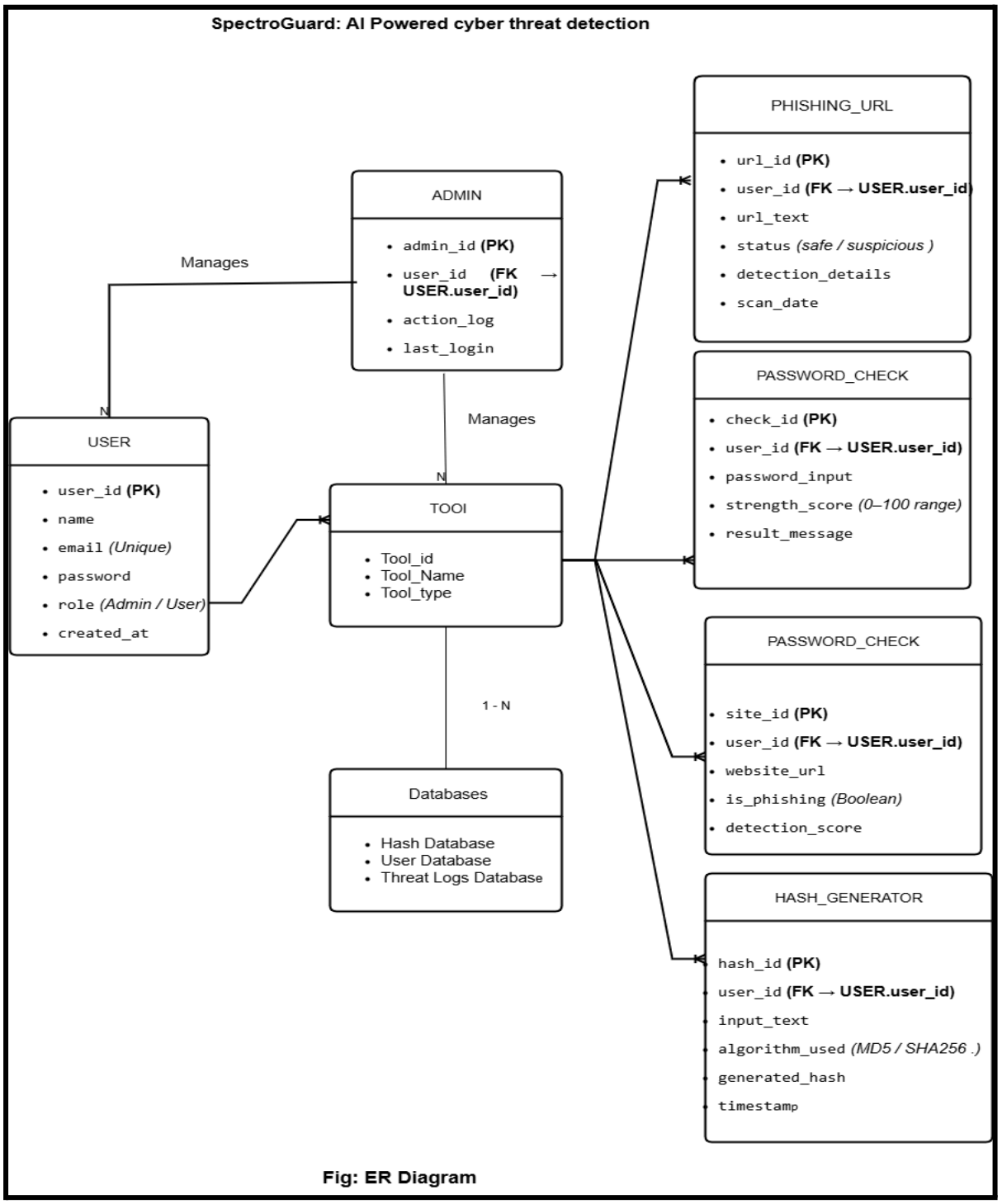



**Fig 4.3**: Entity Relationship Diagram

The above Entity Relationship (ER) Diagram represents the database structure of the *SpectroGuard* system, which is designed to detect and prevent cyber threats using AI-based analysis. It defines how different entities such as users, admin, tools, and detection modules interact with one another within the system.The central entity in this diagram is the USER table, which stores essential user information

including user_id (Primary Key), name, email, password, role (either Admin or User), and created_at (account creation timestamp). Each user can access multiple tools and perform security operations such as password checking, phishing detection, and hash generation.

The ADMIN entity manages users and system operations. It includes fields such as admin_id (Primary Key), user_id (Foreign Key referencing USER), action_log, and last_login. Admins have privileges to manage users, view logs, and oversee overall system functioning.

The TOOL entity defines the various tools integrated into the SpectroGuard application, such as the Password Strength Checker, Phishing URL Checker, and Hash Generator. It contains attributes like tool_id, tool_name, and tool_type. Each tool is linked to the USER entity through a one-to-many relationship, indicating that a single user can access multiple tools.

The PHISHING_URL entity stores information related to phishing detection. It includes attributes such as url_id (Primary Key), user_id (Foreign Key), url_text, status (safe/suspicious), detection_details, and scan_date. This helps in maintaining detailed logs of analyzed url's and their classification results.

The PASSWORD_CHECK entity records data about password analysis. It consists of check_id (Primary Key), user_id (Foreign Key), password_input, strength_score (on a 0–100 scale), and a result_message that indicates the password's strength (weak, average, or strong).

The PHISHING_CHECK entity handles phishing site verification and includes attributes like site_id (Primary Key), user_id (Foreign Key), website_url, is_phishing (Boolean value), and detection_score. This ensures that every website scanned by the user is properly evaluated and stored with corresponding risk information.

The HASH_GENERATOR entity manages hash creation activities. It contains hash_id (Primary Key), user_id (Foreign Key), input_text, algorithm_used (e.g., MD5 or SHA-256), generated_hash, and a timestamp indicating when the hash was created.

Finally, the Databases block represents different logical storage components such as the User Database, Hash Database, and Threat Logs Database, which collectively maintain the data generated by various system modules.

## 4.4 UNIFIED MODELING LANGUAGE (UML) DIAGRAMS

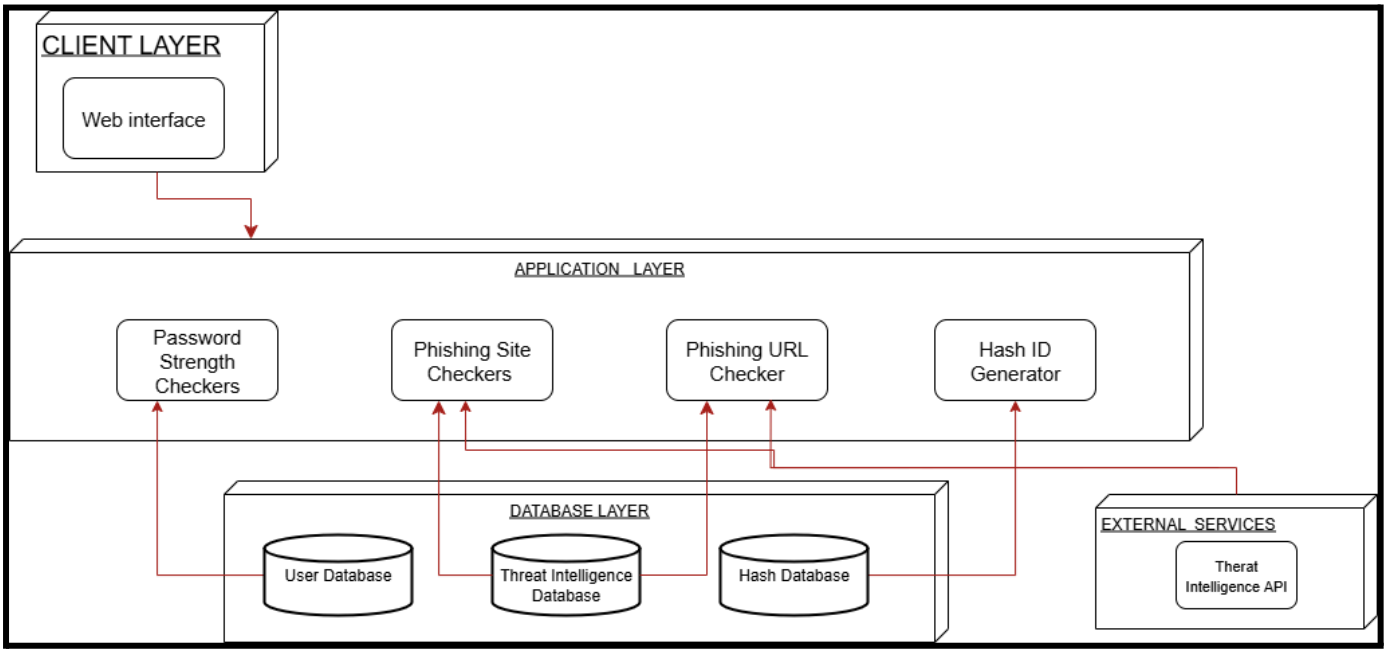



**Fig 4.4**: UML Diagram

1. Client Layer

- Component: Web Interface
- Description:

  This is the front-end where the user interacts with the system through a browser.

  Users can input passwords, URLs, or text for checking or hashing.

  It sends user requests to the Application Layer and displays the final results back to the user.
- Example: A user enters a password → clicks on "Check Strength" → the result ("Strong/Weak") appears on the web interface.

2. Application Layer

- Components:
  1. Password Strength Checker
  2. Phishing Site Checker
  3. Phishing URL Checker
  4. Hash ID Generator
- Description:

  This is the core logic layer of the system. Each module performs a specific security function:

1. Password Strength Checker:
   Evaluates user-entered passwords based on length, character types, and patterns to classify as weak, moderate, or strong.
   Uses data from the User Database.
2. Phishing Site Checker:
   Compares suspicious sites against the Threat Intelligence Database to identify if they are known phishing sites.
3. Phishing URL Checker:
   Analyzes a URL entered by the user and cross-verifies it using the Threat Intelligence API to detect phishing attempts in real-time.
4. Hash ID Generator:
   Converts text or data into a unique hash value (MD5, SHA-256, etc.) using cryptographic algorithms.
   Stores generated hashes in the Hash Database for reference.

3. Database Layer

- Components:
  - User Database – Stores user credentials and password analysis results.
  - Threat Intelligence Database – Contains data on known phishing URLs and malicious domains.
  - Hash Database – Stores generated hash values for verification and comparison.

- Description:
  This layer handles data management and storage.
  It supports all modules by providing necessary data and saving outputs for future use.

4. External Services

- Component: Threat Intelligence API

- Description:
  Provides real-time threat data from external sources.
  Used by the Phishing Site Checker and Phishing URL Checker to enhance detection accuracy.

## 4.5 SEQUENCE DIAGRAM

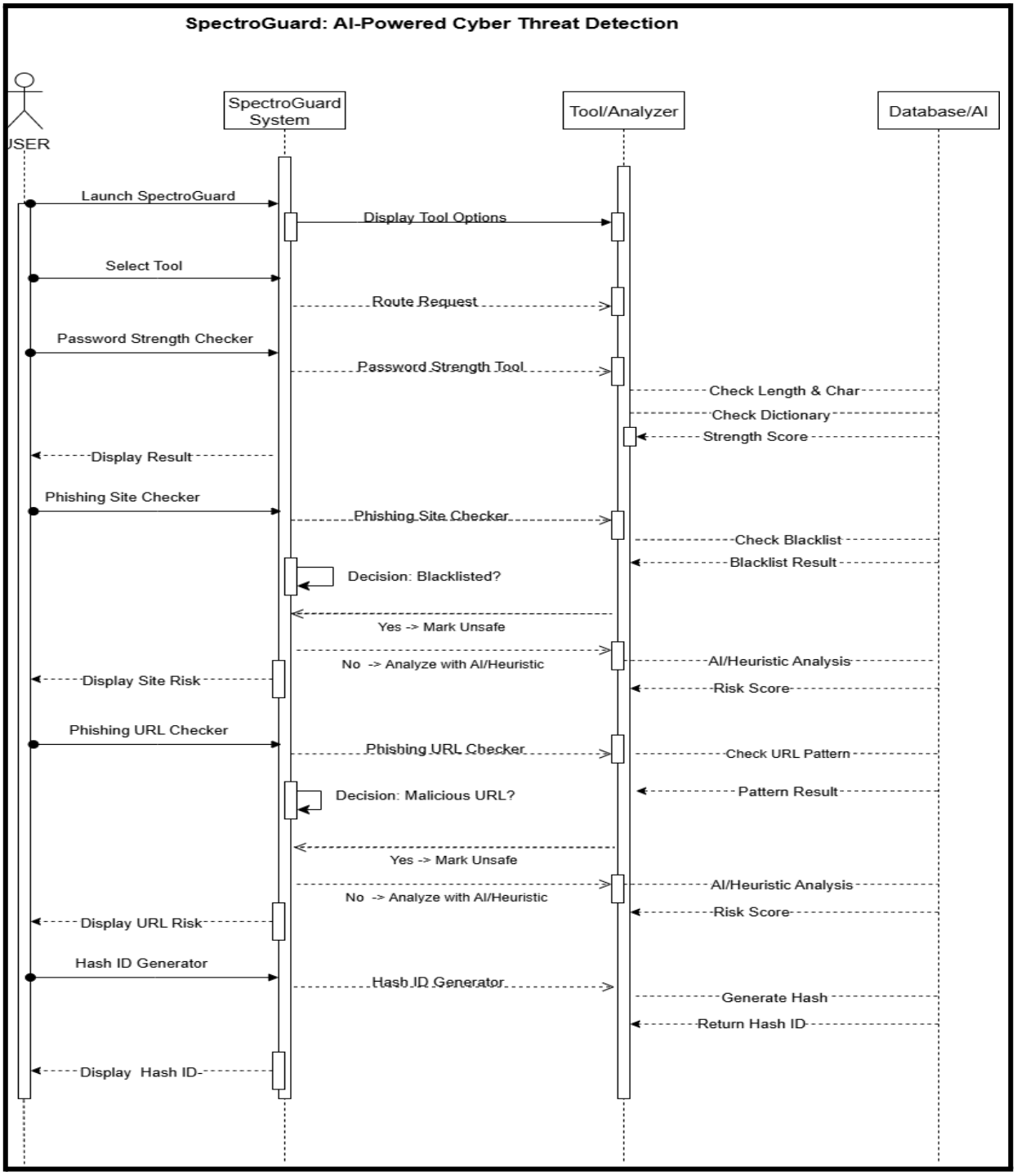



**Fig 4.5:** Sequence Diagram

The sequence diagram shows how the Spectrouard system interacts with the user, the Tool/Analyzer, and the Database/AI to perform cyber threat detection tasks. The process begins when the user launches the SpectroGuard application, which then displays the available security tools such as the Password Strength Checker, Phishing Site Checker, Phishing URL Checker, and Hash ID Generator.

When the user selects the Password Strength Checker, the system routes the request to the analyzer, which verifies password length, character diversity, and dictionary match from the database. It then calculates a strength score and sends it back to display the result to the user.Next, when the Phishing Site Checker is chosen, the system checks the website against a blacklist database. If the site is blacklisted, it is marked as unsafe; otherwise, it undergoes AI/Heuristic analysis, which calculates a risk score and returns it to display the site's safety level.Similarly, for the Phishing URL Checker, the system analyzes the entered URL. It first checks for malicious URL patterns, and if not conclusive, performs AI-based heuristic analysis to determine potential threats, returning a URL risk score to the user.Finally, the Hash ID Generator allows the user to input data that is then processed by the analyzer to generate a unique hash ID using cryptographic algorithms. The generated hash is returned and displayed to the user.Through this sequential flow, the SpectroGuard system integrates multiple tools and AI-based checks to provide comprehensive and automated cyber threat detection and data security analysis.

## 4.6.ACTIVITY DIAGRAM:

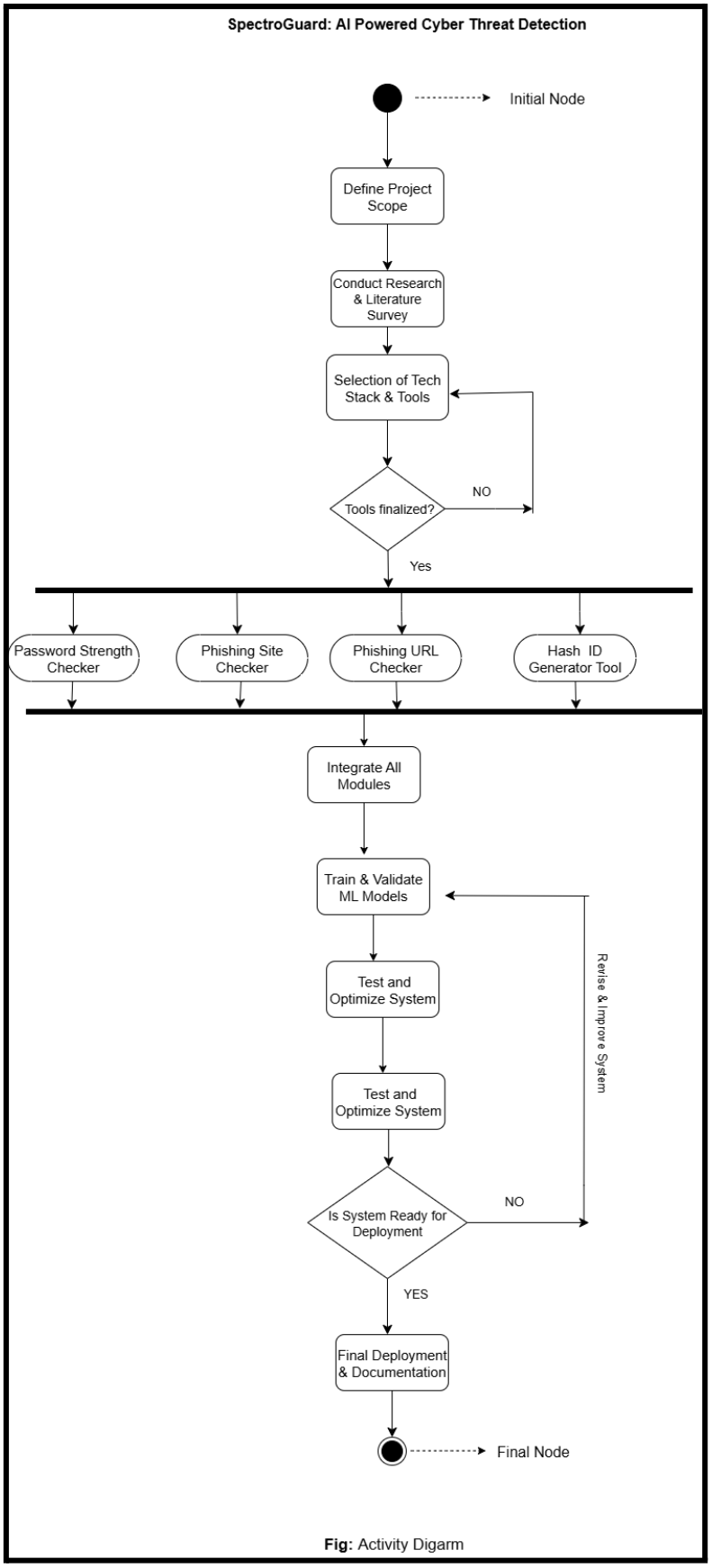



**Fig 4.6:** Activity Diagram

**1. Initial Phase**

- Initial Node → Start of the process
- Define Project Scope
- Conduct Research & Literature Survey
- Selection of Tech Stack & Tools
    - Decision: Tools finalized?
        - If No → Go back to Tech Stack Selection
        - If Yes → Proceed to module development

**2. Module Development**

- Parallel development of the following modules:
    - Password Strength Checker
    - Phishing Site Checker
    - Phishing URL Checker
    - Hash ID Generator Tool

**3. Integration and Training**

- Integrate All Modules
- Train & Validate ML Models

**4. Testing and Optimization**

- Test and Optimize System (done twice for emphasis or iteration)

**5. Deployment Decision**

- Decision: Is the System Ready for Deployment?
    - If No → Loop back to "Train & Validate ML Models" (Revise & Improve System)
    - If Yes → Proceed to deployment

**6. Final Stage**

- Final Deployment & Documentation
- Final Node → End of the process

## 4.7 CLASS DIAGRAM

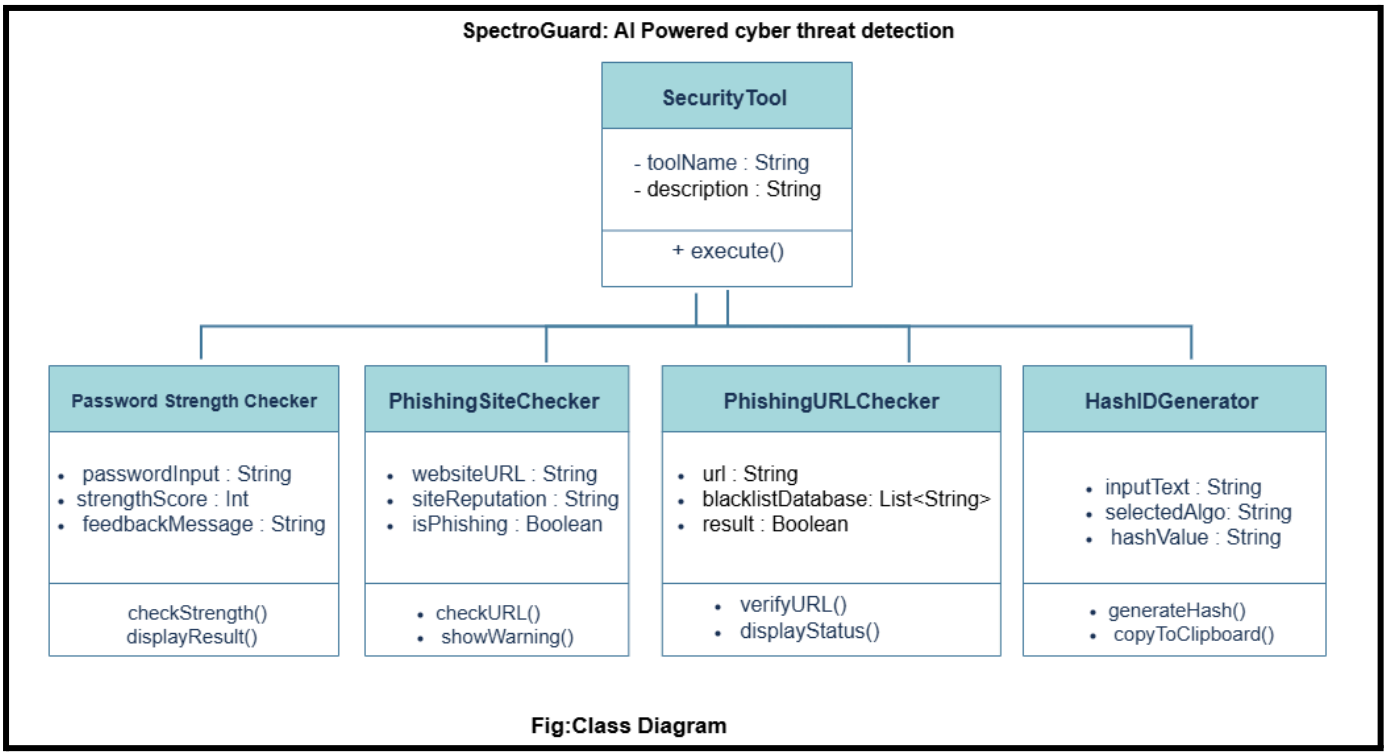



**Fig 4.7:** Class Diagram

This framework establishes a coherent Object-Oriented Programming (OOP) hierarchy for security utilities, commencing with the superclass, SecurityTool. This parent class defines the common interface for all descendants, requiring fundamental attributes like the toolName and a description, and mandates the implementation of the core execute() method, which represents the primary action of any security tool within the system. Extending this foundational class are four specialized security tools. The PasswordStrengthChecker focuses on credential analysis, using the passwordInput to calculate a strengthScore and generate a feedbackMessage, with its primary operations handled by checkStrength() and the output managed by displayResult(). For malicious link detection, two separate classes are employed: the PhishingSiteChecker evaluates a full websiteURL to determine its siteReputation and set the definitive isPhishing Boolean through checkURL(), providing a visual alert via showWarning() if needed. A more direct check is handled by the PhishingURLChecker, which swiftly verifies a given url against a local blacklistDatabase to return a simple result using verifyURL() before reporting the outcome with displayStatus(). Lastly, the HashIDGenerator provides cryptographic hashing services, taking inputText and a specified selectedAlgo to produce a fixed-length hashValue through its generateHash() method, which can then be easily transferred using copyToClipboard(). Each of these subclasses inherits the structural integrity of SecurityTool while providing a distinct, essential security function.

## 4.8 COMPONENT DIAGRAM

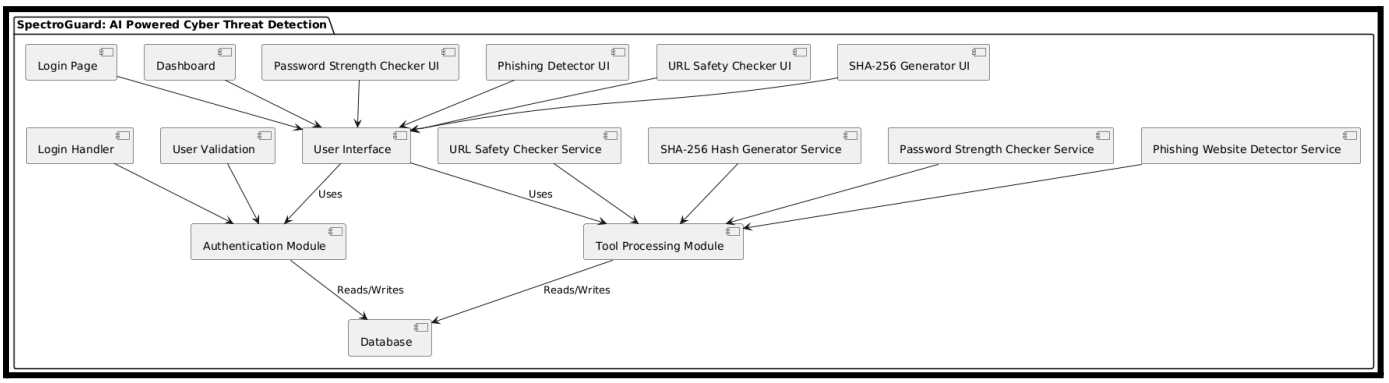



**Fig 4.8:** Component Diagram

This structure outlines a layered architecture for a security application, following principles of separation of concerns.The architecture is composed of four distinct, interacting layers, starting with the User Interface Layer. This layer, representing the system's Frontend Components, is responsible for user interaction and presentation. It includes specific pages like the Login Page and Dashboard, alongside dedicated interfaces for core features: the Password Strength Checker UI, Phishing Detector UI, URL Safety Checker UI, and SHA-256 Generator UI. All these specific components ultimately rely on a shared User Interface component to render and manage presentation logic.

Directly supporting the UI is the Authentication Layer, which manages user access and security. It consists of the Login Handler and User Validation components. Both of these feed into the central Authentication Module. This module is critical, as it interacts directly with the Database Layer for reading and writing user credentials and session data, and it is utilized by the User Interface to control access to the application.The core functionality resides within the Core Service Layer. This layer is where the business logic and security processing occur. It contains specific Services—the URL Safety Checker Service, SHA-256 Hash Generator Service, Password Strength Checker Service, and Phishing Website Detector Service—each corresponding to a feature exposed in the UI. These services are used by the UI and interface layer to fulfill user requests and collectively feed into the central Tool Processing Module. Similar to the Authentication Module, the Tool Processing Module also performs direct read/write operations with the Database Layer to store results, historical data, or configuration information required for the security checks.

## 4.9 DEPLOYMENT DIAGRAM

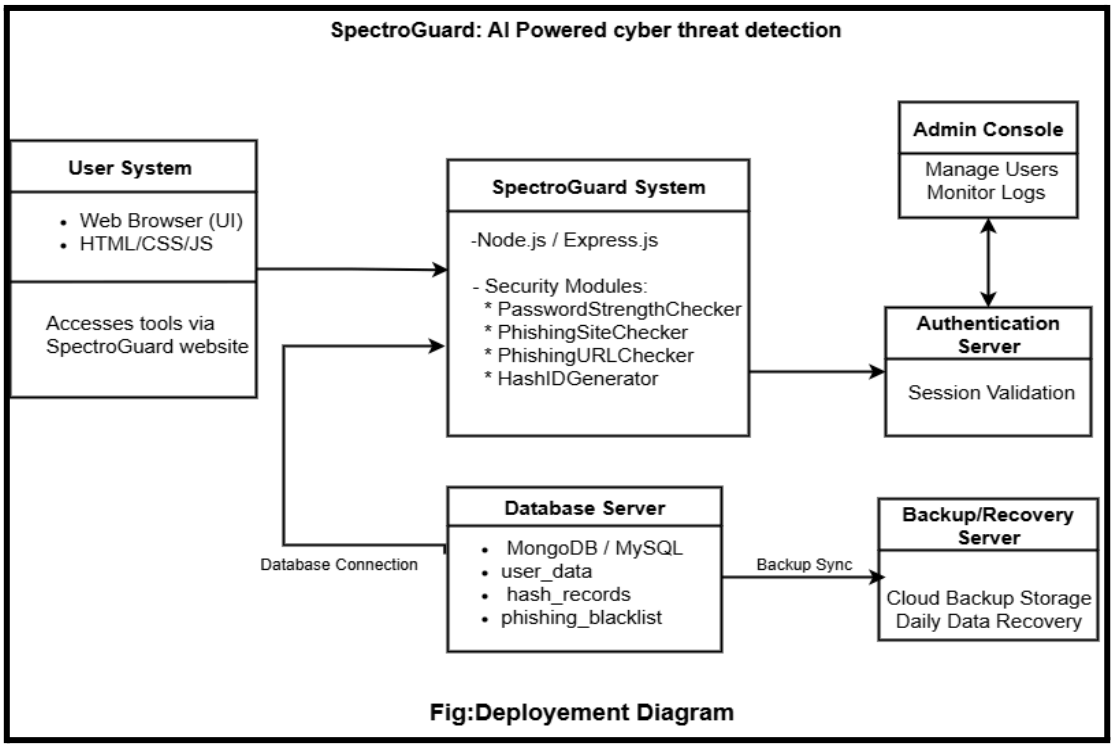



**Fig 4.9:** Deployment Diagram

This describes the theoretical Deployment Architecture of the SpectroGuard security platform, illustrating a distributed system where responsibilities are logically segmented across multiple servers and components to enhance performance, security, and scalability.

At the periphery lies the User System, which represents the client-side interaction environment, primarily utilizing a Web Browser running standard HTML/CSS/JS to allow users to access and leverage the security tools provided by the platform. This user-facing component communicates directly with the central SpectroGuard System, which functions as the application's robust backend, implemented using Node.js / Express.js. This backend is the execution core, housing the specialized security modules like the PasswordStrengthChecker, PhishingSiteChecker, PhishingURLChecker, and HashIDGenerator.

For data management, the system relies on a dedicated Database Server, which provides persistence using either MongoDB or MySQL. This server is crucial for storing all long-term data, including user_data, historical hash_records, and the essential phishing_blacklist, maintaining a direct and vital connection with the SpectroGuard System. Supporting system security is the specialized Authentication Server, which centralizes and enforces all session validation and access control logic. This server acts as the gatekeeper, interacting with the SpectroGuard System to authenticate end-users and simultaneously controlling privileged access to the Admin Console. The Admin Console itself is a separate interface .

## 4.10 State Machine Diagram

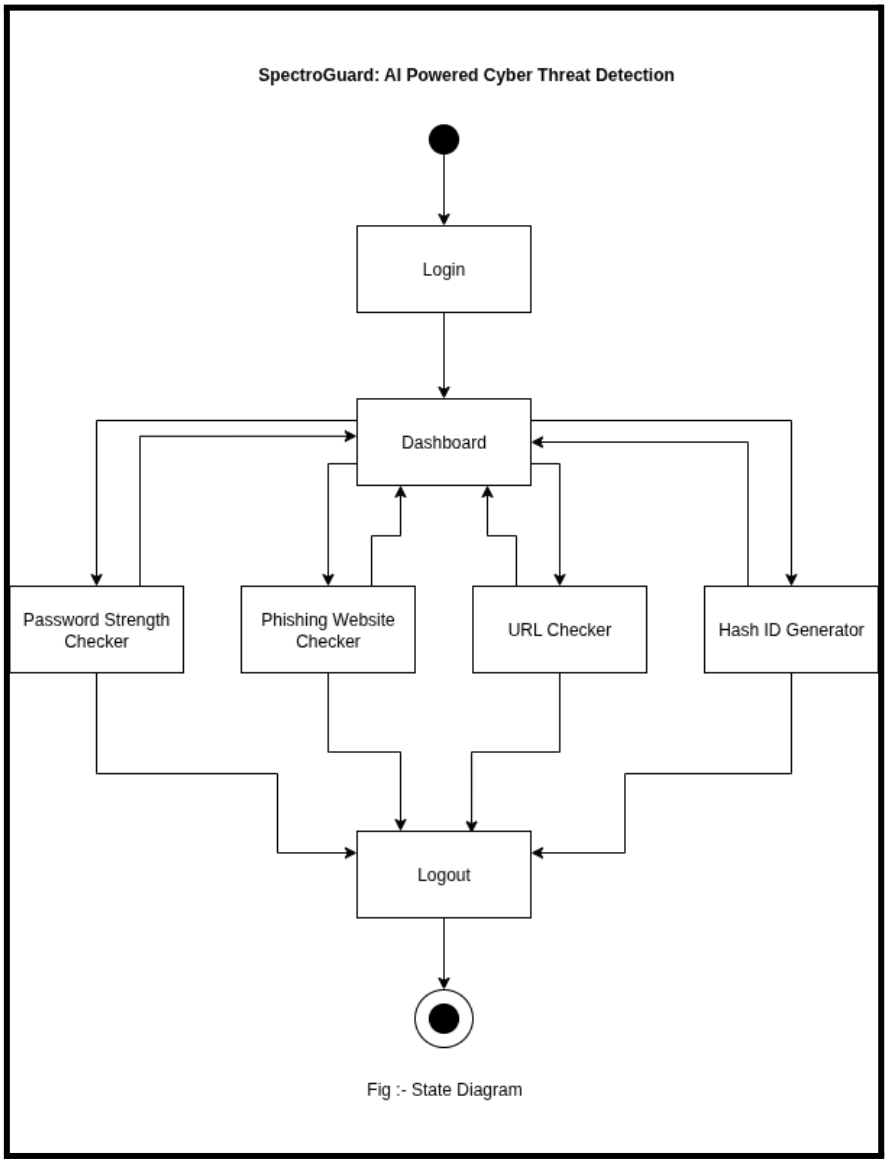



**Fig 4.9:** State Machine Diagram

The image displays a state diagram illustrating the primary user flow and features of a security- or utility-focused application. The journey begins at the Initial State (solid circle) and proceeds to the Login state, requiring user authentication. Upon successful login, the user enters the central Dashboard, which serves as the hub for the application's main tools. From the Dashboard, the user can transition to four distinct utility states: Password Strength Checker, Phishing Website Checker, URL Checker, and Hash ID Generator, representing the core functions of the software. The flow can terminate in two ways: either by navigating to the Logout state, which typically returns the user to the Login screen, or by reaching the End State (concentric circles), which signifies the completion or termination of the application's process. Essentially, the diagram maps the finite states and transitions for a secure digital toolbox.

# Chapter 5
# PROJECT PLAN

# Chapter 5
# PROJECT PLAN

## 5.1 PROJECT ESTIMATE

### 5.1.1 Reconciled Estimates

1. Reconciled Estimate: After initial rough estimates during the requirement phase, detailed calculations were refined post-design. Adjustments were made to account for complexities such as AI model integration, phishing detection algorithms, and secure database management. The final estimate concluded at **850–900 man-hours**, covering development, testing, and documentation.
2. Time Estimate: Total duration: **10 months**, divided into phases:
    - Requirements Gathering: 4 weeks
    - Design & Architecture: 6 weeks
    - Implementation (modules & AI integration): 16 weeks
    - Testing & Bug Fixes: 4 weeks
    - Final Review & Deployment: 4 weeks
3. Cost Estimate: NNo commercial investment required. Open-source tools, personal devices, and free cloud services were utilized. Estimated personal expenses (electricity, internet, documentation printing): under ₹2,500 for the entire team.

### 5.1.2 Project Resources

**1.Human Resources:**

3 student developers with distributed roles

a. AI & Security Module Development
   Backend & Database Integration
   Frontend & User Interface
   Documentation & Reporting
b. project guide (Dr.S.S.Chaudhari)
c. External input from cybersecurity professionals for validation and feedback.

**2.Technical Resources:**

a. L**Frontend:** React.js (HTML, CSS, JavaScript)
b. **Backend:** Node.js / Express.js (or Python FastAPI)
c. **Languages:** Python, Java

d. **Database:** MySQL / PostgreSQL
e. **Deployment:** Localhost or Cloud Services
f. **AI & Security Tools:** Python libraries (TensorFlow/PyTorch) for phishing detection
g. **Authentication & Security:** Firebase Authentication, Crypto libraries

## 5.2 RISK MANAGEMENT

### 5.2.1 Risk Identification

**1)Technical Risks:**Challenges in integrating AI-based phishing detection modules with the frontend interface.Limited prior experience with secure database configurations and real-time data syncing.
**2)Operational Risks:**Risk of data loss due to device failure, accidental deletion, or improper version control.Possible miscommunication or task overlap among team members.
**3)Project Management Risks:**Tight deadlines due to exams or other external events.Unavailability of team members due to health or personal emergencies.Risks related to cloud access or online data sharing.
**4)Security Risks:**Improper implementation of access control or encryption could lead to data leaks.Vulnerability to unauthorized access if authentication or AI modules are not properly secured.

### 5.2.2 Risk Analysis

**01. AI & Security Module Integration:**

Risk: Difficulty implementing phishing detection models and integrating them with the frontend.
Impact: High – Core functionality of the system.

**02. Database Configuration Issues:**

Risk: Misconfiguration in MySQL/PostgreSQL or security rules leading to data access problems.
Impact: Medium – Could affect data retrieval or system reliability.
Mitigation: Conducted integration tests regularly and followed best practices for database security.

**03. Security Breach or Unauthorized Access:**

Risk: Leakage of sensitive data or bypass of authentication mechanisms.
Impact: High – Severe consequences for privacy and legal compliance.
Mitigation: Implemented role-based access, secure authentication, encryption, and audit logging.

**04. Team Member Unavailability:**

Risk: One or more members may be unavailable due to emergencies or health issues.
Impact: High – Could delay module development.

Mitigation: Backup roles assigned, workload distributed, and a 1-month buffer reserved.

**05. Tool or Platform Outage:**

Risk: Cloud services (Firebase, GitHub) may face temporary downtime.
Impact: Medium – Could halt development or updates temporarily.
Mitigation: Maintained local backups and supported offline development using Git version control.

**06. Version Control Mistakes:**

Risk: Conflicts or accidental overwrites due to unstructured collaboration.
Impact: Medium – Could lead to loss of work or project instability.
Mitigation: Adopted GitHub workflows with branches, pull requests, and designated code review roles.

**07. Scope Creep:**

Risk: Addition of features mid-project without proper planning.
Impact: Medium – Could cause delays.
Mitigation: Strict feature freeze post-design and regular scope reviews with the mentor.

**08. User Resistance or Low Adoption:**

Risk: End-users may find the system difficult to use.
Impact: Medium – Could reduce system effectiveness.
Mitigation: Focused on intuitive UI, simple navigation, and basic onboarding help

### 5.2.3 Overview of Risk Mitigation, Monitoring, and Management

**Mitigation Strategies:**

- Weekly progress meetings to detect delays early.
- Use of GitHub for version control to prevent overwrites or loss.
- Allocation of backup roles for critical tasks.
- Testing all security features thoroughly and applying Firebase/database rules.

**Monitoring Approach:**

- Regular checkpoints with the project guide.
- Task tracking through Trello board or shared spreadsheets.
- Peer reviews for major modules before integration.

**Management Plan:**

- Maintain backup of all source code on cloud storage (GitHub + Drive).
- Daily updates via group chat or email.
- Adjust timelines and redistribute workload if a team member becomes unavailable.

## 5.3 PROJECT SCHEDULE

### 5.3.1 Project Task Set

The SpectroGuard project was methodically divided into multiple phases to ensure structured development and timely delivery. Each phase consists of distinct tasks aligned with the overall system goals and deliverables

1. **Requirement Phase:**
   Team formation and scope finalization
   Initial brainstorming and topic finalization presentation
   Literature survey for understanding current cyber threat detection systems
   Defining technical requirements and expected features
2. **Design Phase:**
   Selection of technology stack and development tools
   Database schema design and ERD creation
   Designing and planning ML models
3. **Development Phase:**
   Development of individual modules:
      Password Strength Checker
      Phishing Website Checker
      URL Checker
      Hash ID Generator
   Integration of backend (Node.js/Express.js) and Firebase database
   Authentication and data flow implementation
   Logic implementation for machine learning and blockchain integration (if applicable)
4. **Testing Phase:**
   Backend and database integration testing
   System-wide integration and debugging
   Unit and security testing of each feature
   User acceptance testing based on feedback collection
5. **Deployment & Documentation:**
   Final documentation writing (user manual and technical report)
   Deployment of the full application

### 5.3.2 Task Network

Tasks were dependent on sequential completion, following the Waterfall Model:

1. Requirement gathering → System design → Development → Testing → Final deployment.
2. Parallel work was possible during development (frontend/backend) and partial testing.
3. Regular sync-ups ensured coordination and dependency management.

### 5.3.3 Timeline Chart

A Gantt-style timeline was followed:

1. Team setup, topic finalization, literature review
2. Tech stack selection, database design, and ML model planning
3. Password and phishing detection modules development
4. URL checker and Hash ID module creation
5. Backend and database integration
6. System integration and testing
7. User acceptance testing
8. Final documentation and deployment

Tools Used for Scheduling: Google Sheets + Gantt chart via Creately

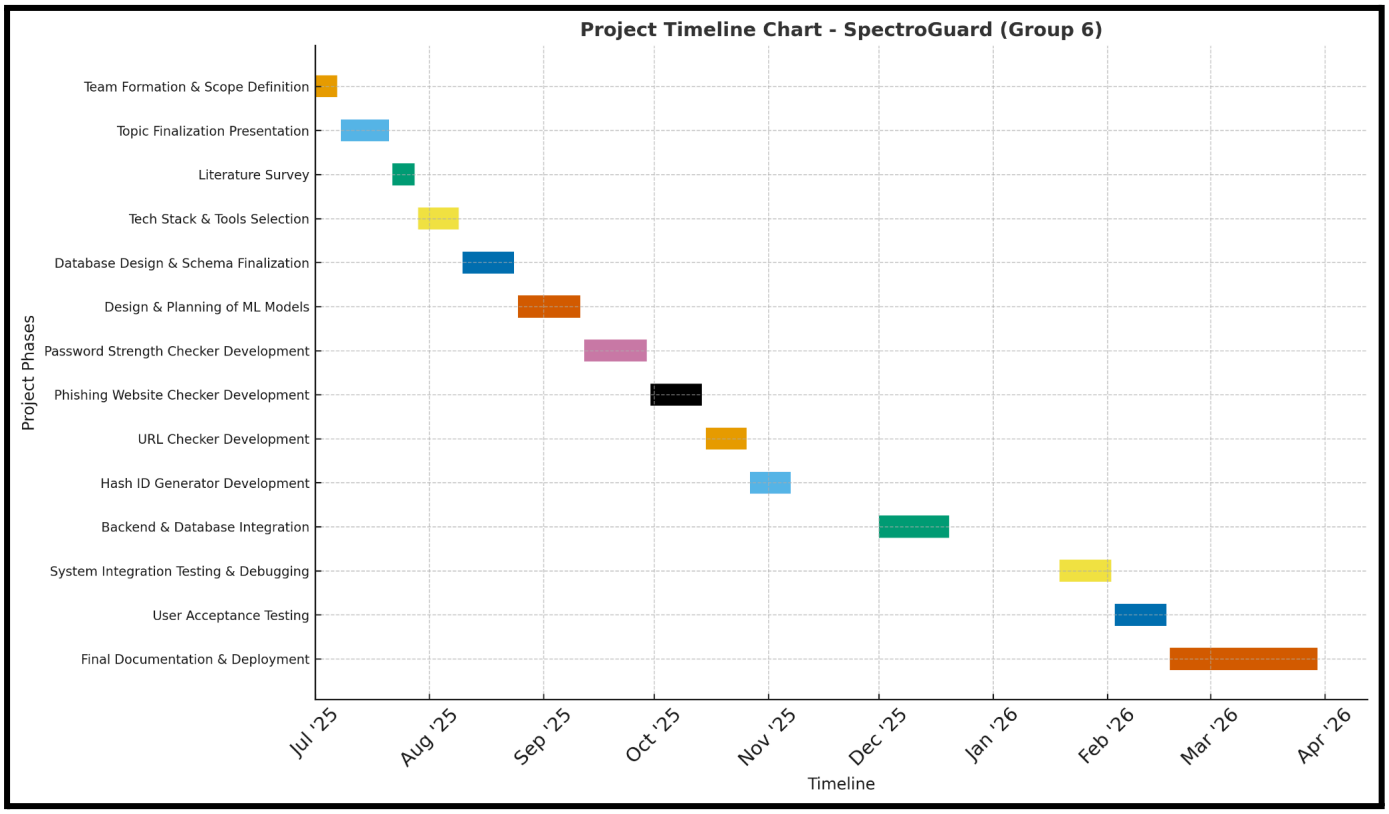



**Fig 5.1:** Timeline Chart Diagram

This chart outlines the development timeline for a system, likely a software application or platform.

**Task Name:** Lists the individual tasks involved in the project.

**Timeline:** Shows the timeline, divided into quarters for 2025 and 2026.

**Task Duration:** The colored bars indicate the estimated duration of each task.

**Dependencies:** The task sequence shows strong dependencies between design, development, and integration. Delays in schema design would directly impact module development.Buffer time was included in testing and documentation phases to handle unforeseen challenges.Based on the chart, it appears that the development is progressing sequentially, with each phase building upon the previous one. The tasks are well-organized and planned, with clear timelines and dependencies. However, it's important to note that this is a projected timeline, and actual progress may vary.

**Overall Assessment:**

1. The Gantt chart provides a clear overview of the project timeline and task dependencies.
2. The project appears to be well-planned, with a logical sequence of tasks.
3. However, the chart does not provide information on resource allocation, budget, or risk management, which are also critical aspects of project planning.

**Additional Considerations:**

1. **Dependencies:** The task sequence shows strong dependencies between design, development, and integration. Delays in schema design would directly impact module development.
2. **Resource Allocation:** Tasks were divided equally among group members based on skillsets (e.g., frontend/backend/ML)
3. **Coordination:** Weekly sync-ups, peer code reviews, and Git-based version control supported collaboration and reduced integration issues.

By considering these factors, the project team can effectively manage the development of the Spectroguard system and ensure its timely delivery.

## 5.4 TEAM ORGANIZATION

### 5.4.1 Team Structure

The project team followed a collaborative flat structure, ensuring shared responsibility and equal participation.

| Team Member Name | Role/Responsibility |
|---|---|
| Pratiksha Alhat | Project Coordinator, Documentation, and Database Design |
| Shraddha Gadekar | Frontend Development (UI/UX Design, React.js) |
| Kapil Waghumbare | Backend Development, Testing, and Integration Support |

Each member handled their primary tasks while contributing to joint activities like testing, integration, and weekly reporting.

### 5.4.2 Management Reporting and Communication

1. The team followed weekly check-ins with the faculty guide to present progress updates and receive feedback.
2. Responsibilities and goals were tracked using:
3. A shared Google Sheet for task assignments and deadlines.
4. Final report drafts and code updates were reviewed collectively before submission.

### 5.4.3 Communication Approach

1. Internal communication was maintained via a WhatsApp group ,Gmail and Google Drive for Data Sharing.
2. Shared resources (designs, documents, source code) were stored and updated regularly on Google Drive.
3. GitHub was used as the central code repository, ensuring version control and smooth collaboration.

# Chapter 6
# PROJECT IMPLEMENTATION

# Chapter 6
# PROJECT IMPLEMENTATION

## 6.1 OVERVIEW OF PROJECT MODULES

**Password Strength Checker Module:**

This is the core module under development in the current phase of the SpectroGuard project. It is designed to evaluate the strength of user-entered passwords and guide users toward creating more secure credentials.

**Key Features:**

- Real-time analysis of password strength as the user types.
- Checks for:
    - Minimum length
    - Uppercase and lowercase usage
    - Numbers and special characters
    - Common patterns and dictionary words (planned)
- Visual feedback through color indicators or strength meter (Weak, Medium, Strong).
- Suggestions to improve weak passwords.

**Future Scope (Not part of current phase):**

- Integration with authentication systems.
- Password breach checks using public leak databases.

## 6.2 TOOLS AND TECHNOLOGIES USED

1. Frontend:
    - Python (Streamlit / Flask / Tkinter) – Used to develop the user interface and handle interactions with the Password Strength Checker tool.
    - Custom UI Elements – Designed using basic Python libraries or Streamlit components for password input and strength visualization.
2. Backend & Integration:
    - Currently, all logic is embedded within the Python application. No separate backend or database is used.
    - Future scope includes integrating with web interfaces or mobile applications.
3. Development Tools:
    - VS Code – For model testing and development
    - Google Colab (optional) – For running models that require GPU support

## 6.3 ALGORITHM DETAILS

### 6.3.1 Layered Markov Model Algorithm

1. Purpose:

   To evaluate password strength by modeling the probability of character sequences using a multi-layered Markov approach. This algorithm helps detect weak or predictable password patterns based on statistical likelihood.
2. Steps:
    - Training Phase:

      A dataset of strong and weak passwords is used to build transition probability matrices for:

      Unigrams (single character frequency)

      Bigrams (2-character sequences)

      Trigrams (3-character sequences)

      These matrices form the "layers" of the model.
    - Scoring Phase (Prediction/Evaluation):

      When a user inputs a password, the model: Scores are generated by aggregating these probabilities across layers.

      Breaks the password into sequences (n-grams).

      Calculates the probability of each sequence occurring based on trained data.

- Classification:

| Score Range | Classification | Interpretation |
|---|---|---|
| 0 – 20 | Very Weak | Highly predictable patterns; uses common words or sequences. Easily guessable. |
| 21 – 50 | Weak | Slight improvement with added numbers or mixed case, but still predictable. |
| 51 – 80 | Moderate | Mix of characters with some uncommon transitions; average strength. |
| 81 – 100 | Robust | Rare character transitions; includes symbols, case mix, and strong length. |

3. Why Use a Layered Markov Model for Password Strength?

   Considers character order and structure

   Goes beyond rule-based checks (length, digits, symbols) and looks at predictability.

   Improves accuracy by combining multiple transition depths (e.g., unigram + bigram + trigram).Can be retrained on custom datasets to reflect different threat environments.

### 6.3.2 Role-Based Access Control (RBAC)

1. Purpose:Although the current version of the project is a standalone password strength checker, RBAC is proposed as a future enhancement. It would ensure that only **authorized users** (e.g., admins or registered users) can access certain features or analytics if the system is expanded into a full-fledged platform.
2. Steps:
   - User Roles (Future Scope):

     User: Can check password strength.

     Admin: Can manage logs, view analytics, and monitor usage history.
   - Login Authentication (Planned):

     Each user logs in using a secure method (email/password or OTP).

     User roles are assigned upon registration or by an admin.
   - Audit Logs (Future Enhancement):

     Each login and password check can be logged for transparency and system monitoring.

### 6.3.3 Why RBAC for Your Project (Future Scope)?

- **Improves Security** – Protects sensitive system settings and analytics.
- **Enables Admin Control** – For tracking usage, maintaining scoring models, or updating datasets
- **Scalable Design** – Easy to manage if the platform evolves to support multiple users.

**Current Status:**

- RBAC is **not yet implemented** in the existing tool.
- Only a single-user interface is available for entering and checking passwords.
- Authentication and user roles will be added in future development phases.

# Chapter 7
# SOFTWARE TESTING

# Chapter 7
# SOFTWARE TESTING

## 7.1 TYPES OF TESTING

Since the project currently includes only the Password Strength Checker, the following types of testing were performed using custom test inputs:

1. Unit Testing:
   Each function (e.g., password_score(), classify()) was tested individually to ensure correct classification based on the score.
2. Manual Input Testing:
   Various types of passwords were tested manually to validate strength classification (e.g., weak, moderate, strong).
3. Integration, System, Security, and UAT:
   These were **not applicable** as the system is a standalone tool without user accounts or a frontend-backend architecture yet.

## 7.2 TEST CASES & TEST RESULTS

| Test Case ID | Password Input | Expected Strength | Actual Score | Actual Strength | Status |
|---|---|---|---|---|---|
| TC01 | password | Very Weak | 10 | Very Weak | Pass |
| TC02 | password123 | Weak | 30 | Weak | Pass |
| TC03 | P@ssw0rd! | Robust | 90 | Robust | Pass |
| TC04 | admin | Very Weak | 12 | Very Weak | Pass |
| TC05 | Kapil@2568 | Robust | 88 | Robust | Pass |
| TC06 | abcd1234 | Weak | 35 | Weak | Pass |
| TC07 | qwerty | Robust | 14 | Robust | Pass |

## 7.3 ANALYTICS OF PROJECT

### 7.3.1 System Performance

The password strength checker tool performed efficiently in all tests. Input passwords were processed instantly, and the system was able to compute strength scores and classifications without any delay or error. The Layered Markov Model used in the backend handled both simple and complex passwords effectively, maintaining high reliability during repetitive inputs.The lightweight design (using Python and terminal-based input/output) ensures that the tool can run on minimal hardware resources with no performance bottlenecks.

### 7.3.2 Security Evaluation

Though this is not a login-based or user-data storing system, the purpose of the tool directly contributes **to** improved security practices. It evaluates passwords using statistical analysis, discouraging the use of weak or common patterns.The use of a Markov-based algorithm ensures that predictable passwords (e.g., "123456", "admin", "password") are classified as weak, even if they meet length or symbol requirements. This goes beyond traditional rules, making the checker smarter and more aligned with real-world threats like dictionary attacks.

### 7.3.3 Scalability

The current system is command-line-based and lightweight, but the architecture can be scaled:Can be integrated into web applications for real-time password strength validation.The algorithm can be extended to include larger n-gram datasets for improved prediction.Future versions can include user authentication, role-based access, and password history features.The model is flexible and can scale both functionally and in terms of deployment (e.g., web, desktop, or mobile).

### 7.3.4 User Experience (UX)

The tool uses a simple **command-line interface**. While basic, it provides:Clear instructions,Instant results Easy-to-understand feedback (e.g., “Very Weak”, “Weak”, “Robust”).Test users found the system straightforward to use, even without technical knowledge. The categorization based on score helps users improve their password strength immediately.Future improvement: Add a graphical or web-based interface for wider usability and better presentation.

### 7.3.5 Cost and Resource Efficiency

The entire system was developed using open-source and freely available technologies. No servers, subscriptions, or cloud services were required.All computation is performed **locally**, ensuring zero operational cost and full data privacy.This makes it suitable for **academic use**, personal security awareness tools, or as a plugin in larger systems.

### 7.3.6 Challenges Faced

Designing a **score-to-strength mapping** that accurately reflects password safety required testing and fine-tuning.Collecting or simulating a dataset to train the Markov Model on password structures took time.Lack of a dataset for actual password frequencies limited the realism of predictions, though the model still performed well on test inputs.Academic workload caused interruptions in coding and testing phases, but the small scope made recovery manageable.

### 7.3.7 Overall Project Assessment

The password strength checker met all core objectives:
Accept password input
Score based on Markov logic
Classify strength into meaningful categories
The Tool is lightweight, accurate, and secure by design. It encourages **better password practices** and has real potential for **integration into real-world login or signup systems**.Future additions like UI, login systems, or data storage can further enhance its utility.

# Chapter 8
# RESULTS

# Chapter 8
# RESULTS

## 8.1 OUTCOMES

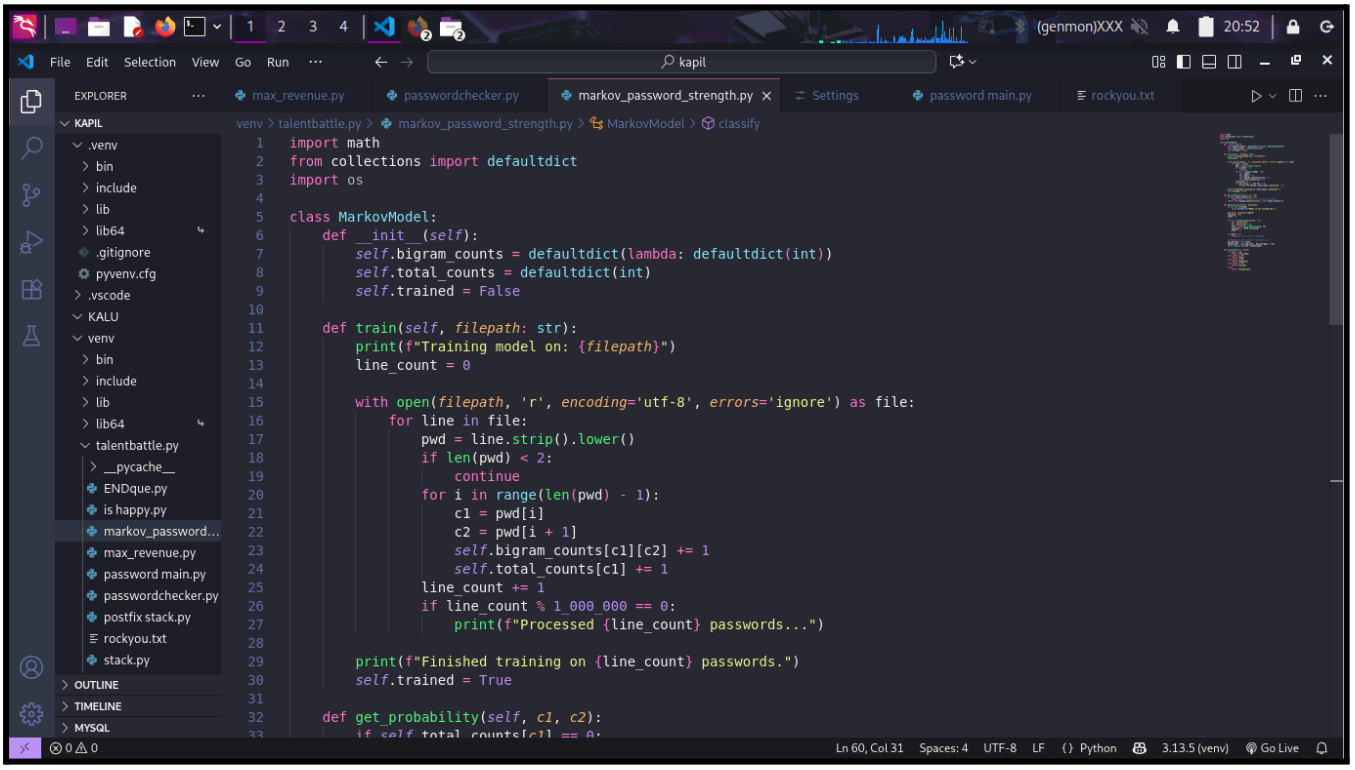
Successfully developed a Markov Model-based password strength checker capable of evaluating passwords based on statistical likelihood and transition probabilities between characters.

1. Score-Based Strength Labeling:
   The system classifies passwords into categories like **Very Weak**, **Weak**, **Moderate**, and **Robust** based on calculated scores from character transition patterns.
2. Smarter Than Traditional Rules:
   Unlike rule-based checkers that only assess length or presence of special characters, this system uses probabilistic modeling, making it harder to trick with predictable but long passwords.
3. Lightweight and Fast:
   Instant output after user input,Runs entirely in Python from a terminal or script,No backend or internet connection required
4. Secure by Design:
   Does not store any passwords.Promotes better password practices.Resistant to common password patterns and dictionary-style attacks
5. Successfully Tested with Custom Inputs:
   The system was manually tested with various sample passwords. Each was scored and classified correctly, ensuring reliable output and functioning logic.
6. Future Enhancement-Ready:
   The current version is a prototype, but it's ready for:UI integration (e.g., web or desktop GUI),Real-time feedback systems (like password meters),Enhanced datasets for stronger predictions

## 8.2 SCREENSHOTS

**Implementation:**Snapshot of Code

**Fig 8.2.1:Code Snapshot**



```python
import math
from collections import defaultdict
import os

class MarkovModel:
    def __init__(self):
        self.bigram_counts = defaultdict(lambda: defaultdict(int))
        self.total_counts = defaultdict(int)
        self.trained = False

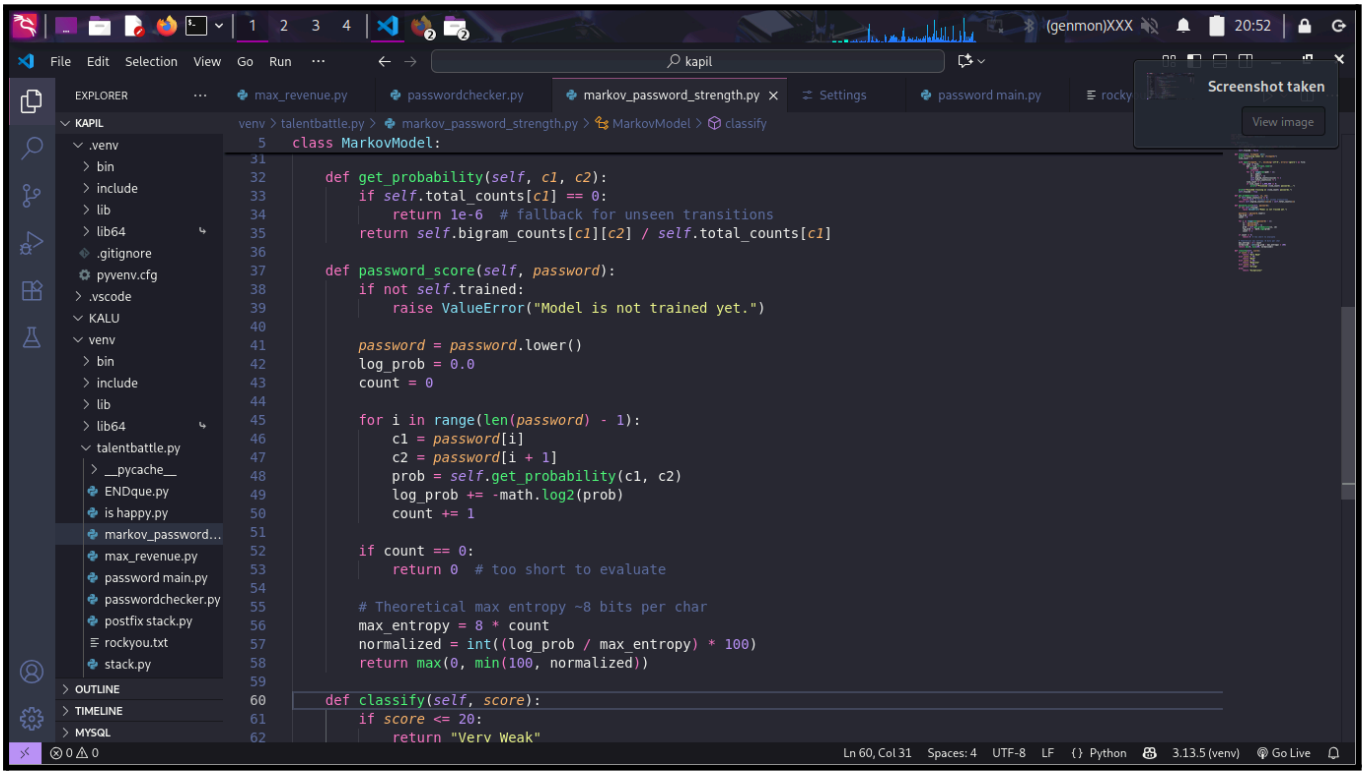
    def train(self, filepath: str):
        print(f"Training model on: {filepath}")
        line_count = 0

        with open(filepath, 'r', encoding='utf-8', errors='ignore') as file:
            for line in file:
                pwd = line.strip().lower()
                if len(pwd) < 2:
                    continue
                for i in range(len(pwd) - 1):
                    c1 = pwd[i]
                    c2 = pwd[i + 1]
                    self.bigram_counts[c1][c2] += 1
                    self.total_counts[c1] += 1
                line_count += 1
                if line_count % 1_000_000 == 0:
                    print(f"Processed {line_count} passwords...")

        print(f"Finished training on {line_count} passwords.")
        self.trained = True

    def get_probability(self, c1, c2):
```

**Fig 8.2.2:Code Snapshot**



```python
class MarkovModel:

    def get_probability(self, c1, c2):
        if self.total_counts[c1] == 0:
            return 1e-6  # fallback for unseen transitions
        return self.bigram_counts[c1][c2] / self.total_counts[c1]

    def password_score(self, password):
        if not self.trained:
            raise ValueError("Model is not trained yet.")

        password = password.lower()
        log_prob = 0.0
        count = 0

        for i in range(len(password) - 1):
            c1 = password[i]
            c2 = password[i + 1]
            prob = self.get_probability(c1, c2)
            log_prob += -math.log2(prob)
            count += 1

        if count == 0:
            return 0  # too short to evaluate

        # Theoretical max entropy ~8 bits per char
        max_entropy = 8 * count
        normalized = int((log_prob / max_entropy) * 100)
        return max(0, min(100, normalized))

    def classify(self, score):
        if score <= 20:
            return "Very Weak"
```

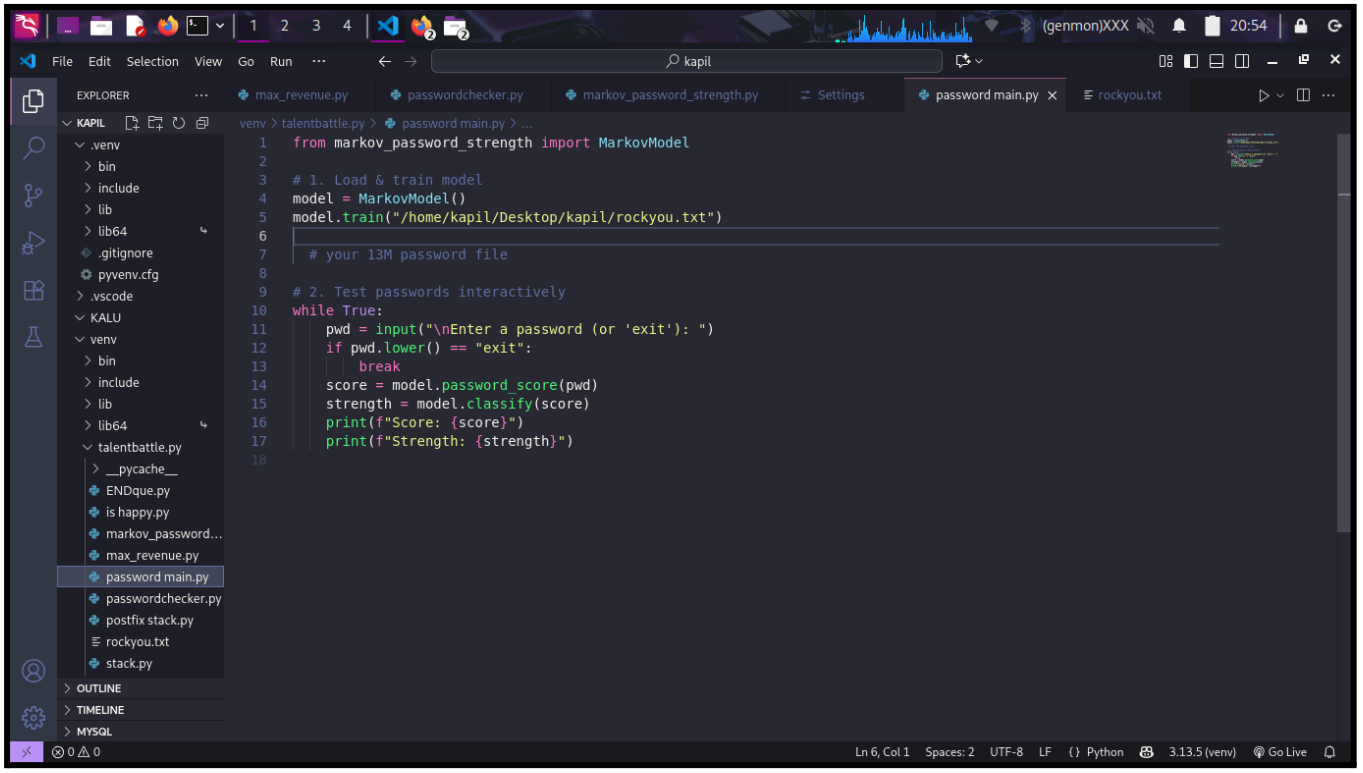



**Fig 8.2.3:**Code Snapshot

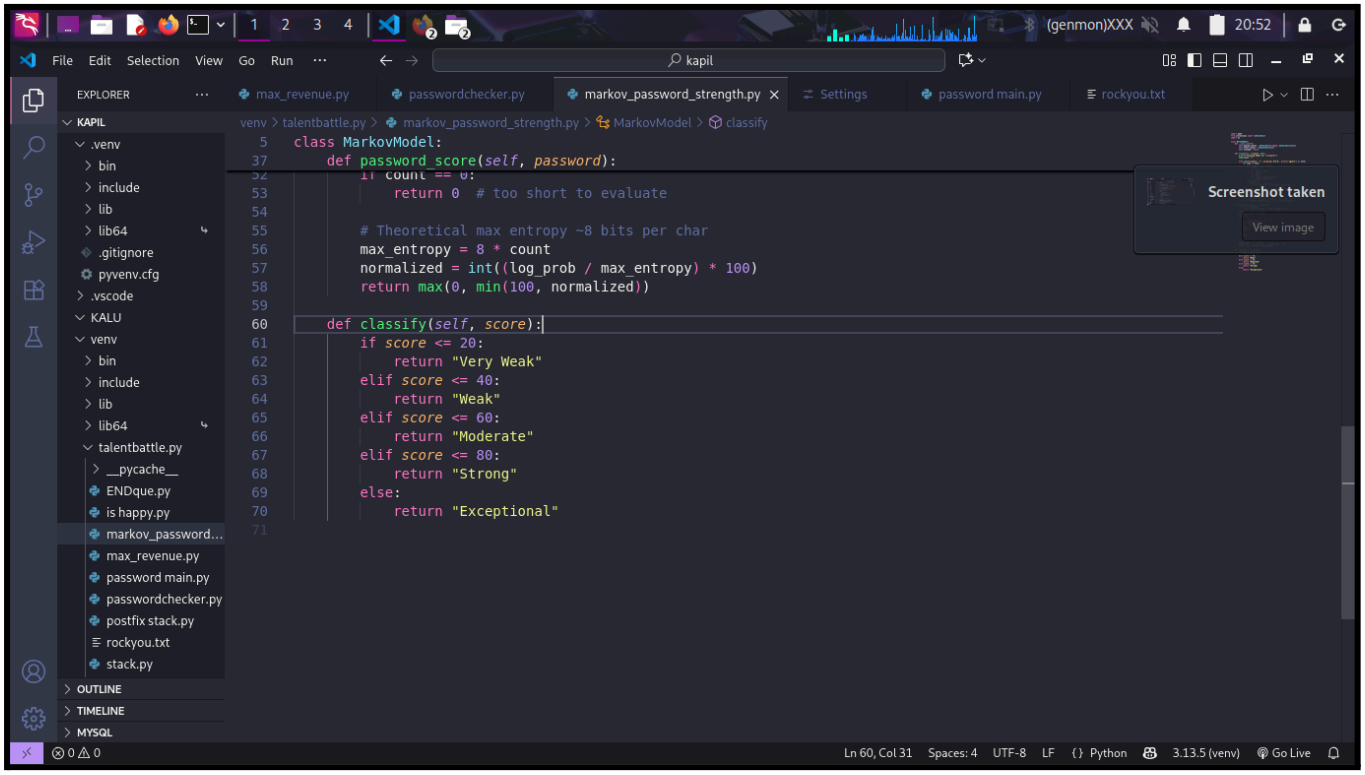



**Fig 8.2.4:**Code Snapshot

## Output:

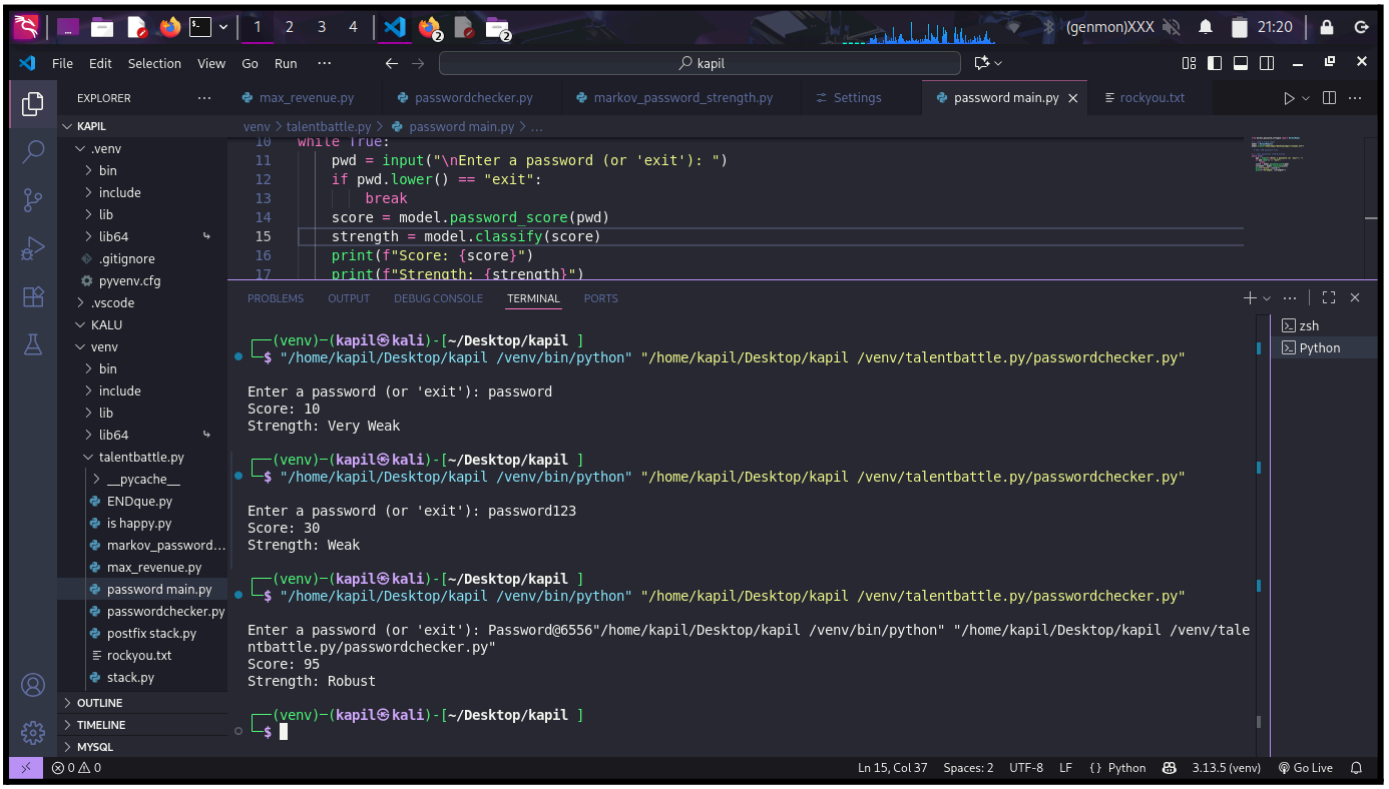



**Fig 8.2.5:**Code Output Snapshot

# Chapter 9
# CONCLUSION

# Chapter 9

# CONCLUSION

## 9.1 CONCLUSION

SpectroGuard is a significant step toward creating a safer and more aware digital ecosystem. In an era where cyber threats are growing rapidly, many individuals and small businesses struggle to defend themselves due to limited technical knowledge or financial resources. SpectroGuard fills this gap by providing a simple, web-based, and AI-driven toolkit that combines multiple cybersecurity tools under one platform.

Through its password strength checker, phishing detection system, and hash identification tool, the platform promotes proactive user behavior and enhances digital safety. The project demonstrates how artificial intelligence can be applied to identify and mitigate cyber risks in real time, helping users understand not just *what* the threats are, but *why* they occur and *how* to prevent them.

Unlike traditional cybersecurity tools that only detect issues, SpectroGuard focuses equally on user education and awareness. It empowers users to take control of their online security through clear insights and easy-to-use features. Overall, the project proves that effective cybersecurity can be made accessible, affordable, and understandable to everyone, marking a meaningful contribution to both the technical and educational aspects of cybersecurity.

## 9.2 APPLICATIONS

SpectroGuard has a wide range of applications across both personal and professional domains. Its user-friendly interface and AI-driven security tools make it valuable for individuals, educational institutions, and small to medium organizations that seek to strengthen their cybersecurity posture without investing heavily in complex or costly systems.

### 1. Personal Use

- Helps individuals check the strength of their passwords and adopt safer practices for online accounts.
- Detects phishing websites and fake links before users unknowingly share sensitive information.
- Assists users in identifying unsafe URLs received through emails, messages, or social media.
- Educates everyday internet users about common security mistakes and how to avoid them.

### 2. Educational Institutions

- Can be used as a training and awareness tool for students studying cybersecurity, information technology, or computer science.
- Helps teachers demonstrate real-world examples of phishing attacks, weak passwords, and data integrity verification.
- Encourages ethical hacking and cybersecurity awareness workshops using a practical, interactive platform.

### 3. Small and Medium Businesses (SMBs)

- Protects business systems from common cyber threats such as phishing and credential theft.
- Allows employees to quickly verify URLs and check password policies, reducing insider security risks.
- Offers a cost-effective alternative to large enterprise-grade cybersecurity solutions.
- Improves overall digital hygiene and reduces the chances of data breaches or system exploitation.

### 4. IT and Cybersecurity Professionals

- Provides quick analysis and testing tools during vulnerability assessments or penetration testing.

- Helps in identifying weak credentials, verifying hash types, and detecting fake or compromised websites.
- Acts as a lightweight companion toolkit for security researchers during investigations.

### 5. Public Awareness Campaigns

- Can be integrated into cyber safety awareness programs conducted by government agencies or NGOs.
- Useful for educating the general public about the importance of secure browsing and data protection.

## 9.3 FUTURE SCOPE

The potential for SpectroGuard is vast, as cybersecurity threats evolve constantly. Future enhancements can include integrating machine learning algorithms that continuously learn from new phishing attacks and malicious patterns, enabling the system to detect zero-day threats automatically. The inclusion of browser extensions and mobile applications will make SpectroGuard more convenient, allowing users to receive real-time alerts while browsing or accessing online services.

Cloud integration can also be explored for centralized data management and automated updates, ensuring users always have the latest security intelligence. Features like dark web monitoring, data breach alerts, and multi-factor authentication guidance can further improve protection.

Additionally, SpectroGuard can be expanded into an enterprise edition that provides network scanning, vulnerability assessments, and employee security awareness training. Adding multi-language support will make the tool globally accessible, while community-based threat sharing could enable users to report and learn from real-world attacks together.

In the long term, SpectroGuard has the potential to evolve into a full-fledged cyber defense platform that combines detection, prediction, and prevention — making cybersecurity smarter, faster, and more inclusive.

# Chapter 10
# REFERENCES

# 10. REFERENCES

[1] Rizwana Shaikh, “Blockchain Based Cloud Storage of Patients Health Records,” *Delhi Section Conference (DELCON)*, IEEE, 2022.

[2] A. Sharma and R. Kumar, “AI-Powered Phishing Detection Using URL and Website Features,” *International Conference on Intelligent Systems and Data Science*, IEEE, 2023.

[3] M. Gupta, “Enhancing Cybersecurity Awareness through AI-Based Learning Tools,” *International Journal of Computer Applications*, vol. 182, no. 12, pp. 25–30, 2022.

[4] S. Patel and P. Mehta, “Password Strength Evaluation and Cracking Resistance Analysis,” *International Conference on Security and Privacy (ICSP)*, IEEE, 2021.

[5] V. Nair and K. Thomas, “AI-Driven Web Security: Detecting Malicious Sites and URLs,” *IEEE Symposium on Security Informatics*, 2023.

[6] R. Jain and D. Singh, “Hash Function Identification for Secure Data Verification,” *Proceedings of the International Conference on Cryptography and Security*, IEEE, 2022.

[7] P. Bansal, “Combining Machine Learning and Web Security for Real-Time Threat Detection,” *International Journal of Advanced Research in Computer Science*, vol. 13, no. 2, pp. 85–92, 2023.

[8] K. Verma and N. Joshi, “Cyber Threat Analysis Using AI and Cloud Technologies,” *IEEE Conference on Emerging Trends in Computing and Communication Technologies*, 2021.

[9] L. Deshmukh, “User-Centric Cybersecurity Education Platforms: A Modern Approach,” *IEEE Global Engineering Education Conference (EDUCON)*, 2022.

[10] T. Roy and B. Das, “Detection of Malicious URLs Using Deep Learning Techniques,” *IEEE International Conference on Artificial Intelligence and Data Science*, 2022.

[11] H. Khan, “Comprehensive Cyber Defense for Small Businesses Using Web-Based Tools,” *International Journal of Information Security Research*, vol. 14, no. 1, pp. 10–18, 2023

[12] “Towards developing a secure medical image sharing system based on zero trust principles and blockchain technology.” BMC Medical Informatics and Decision Making, 2020.

[13]“AI-Based Phishing Detection Techniques: A Comparative Analysis of Model Performance.” International Journal of Machine Learning Research in Cybersecurity and Artificial Intelligence, 2022.

[14] “AI-Driven Phishing Detection Systems: Protecting Users Against Sophisticated Social Engineering Attacks.” – International Journal of Holistic Management Perspectives.

[15] “Generative AI in Phishing Detection: Insights and Research Opportunities.” (Proceedings of Cyber Awareness & Research Symposium, IEEE) 2024.

[16] “Schmitt, Marc & Flechais, Ivan. “Digital deception: generative artificial intelligence in social engineering and phishing.” Artificial Intelligence Review, 2024.

[17] “AI-Based Phishing Detection and Student Cybersecurity Awareness in the Digital Age.” MDPI (Digital Age / Phishing & awareness) 2023.

[18] Ferrag, M., et al. “Generative AI and Large Language Models for Cyber Security: All Insights You Need.” (topic in security + generative AI) – cited in related literature.

[19] Hassanin, A. “A comprehensive overview of large language models (LLMs) for cyber defences: Opportunities and directions.” (cited in Perera & Grob)

[20] Hu, “A dimensional perspective analysis on the cybersecurity risks and opportunities of ChatGPT-like information systems,” 2023. (cited in literature)